



SOFTWARE ENGINEERING 2

Chrisna L. Fucio
Joselle A. Banocnoc



Bachelor of Science in Information Technology
College of Computing Studies

v2.01182024

Table of Contents

Module 5: System Engineering

Introduction	63
Learning Outcomes	64
Lesson 1. Systems	64
Lesson 2. Sociotechnical systems	68
Lesson 3. Conceptual design	74
Lesson 4. System procurement	76
Lesson 5. System development	80
Lesson 6. System operation and evolution	85

Module 6: Component Based Software Engineering

Introduction	89
Learning Outcomes	91
Lesson 1 System complexity	92
Lesson 2 Systems of systems classification	94
Lesson 3 Reductionism and complex systems	95
Lesson 4 Systems of systems engineering	98
Lesson 5 Systems of systems architecture	103

Module 7: Real-time Software Engineering

Introduction	106
Learning Outcomes	108
Lesson 1 Embedded systems design	109
Lesson 2 Architectural patterns for real-time software	115
Lesson 3 Timing analysis	120
Lesson 4 Real-time operating systems	124
Assessment Task	128
Summary	132
References	133

MODULE 5

SYSTEM ENGINEERING



Introduction

A computer only becomes useful when it comprises both software and hardware. Without hardware, a software system is an idea of a simple representation of some human knowledge and ideas. Without software, a hardware system is a set of simple electronic devices. However, they go together to form a computer system, a machine can be created which can carry out complex computations and deliver the results of these computations to its environment (Sommerville, n.d.).

To explain one of the fundamental characteristics of a system: It is more than the sum of its parts. Systems have properties that only become clearly understood when their components are integrated and operate together. Furthermore, systems are developed to support human activities like work, entertainment, communication, protection of people and the environment, and others. They interact with people, and their design is influenced by human and organizational concerns. Hardware, human, social, and organizational factors have to be taken into account when developing all professional software systems (Sommerville, n.d.).

Systems engineering is the activity of designing entire systems, taking into account the characteristics of hardware, software, and human elements of these systems. Systems engineering includes everything to do with procuring, specifying, developing, deploying, operating, and maintaining both technical and sociotechnical systems. Systems engineers have to consider the capabilities of hardware and software as well as the system's interactions with users and its environment. They must think about the system's services, the constraints under which the system must be built and operated, and the ways in which the system is used (Sommerville, n.d.).



Learning Outcomes

At the end of this module, students should be able to:

1. Examine systems engineering and the most important systems engineering processes;
2. Identify sociotechnical system and human, social, and organizational issues which affect the requirements and design of software systems;
3. Assess the idea of conceptual design and why it is an essential first stage in the systems engineering process;
4. Differentiate system procurement and its processes are used for different types of system; and
5. Use systems engineering development processes and their relationships.

Lesson 1. Systems

The term system is universally used. We talk about computer systems, operating systems, payment systems, the education system, the system of government, and so on. These are all obviously quite different uses of the word “system,” although they share the essential characteristic that, somehow, the system is more than simply the sum of its parts.

System

A system is a purposeful collection of interrelated components of different kinds that work together to deliver a set of services to the system owner and its users (Sommerville, n.d.).

This general definition can cover a very wide range of systems. For example, a simple system, such as a laser pointer, delivers an indication service. It may include a few hardware components with a tiny control program in read-only memory (ROM). By contrast, an air traffic control system includes thousands of hardware and software components as well as human users who make decisions based on information from that computer system. It delivers a range of services, including providing information to pilots, maintaining safe separation of planes, utilizing airspace, and so on. In all complex systems, the properties and behavior of the system components are inextricably intermingled.

The successful functioning of each system component depends on the functioning of other components. Software can only operate if the processor is operational. The processor can only carry out computations if the software system defining these computations has been successfully installed (Sommerville, n.d.).

Systems that include software are into two categories (Sommerville, n.d.):

1. **Technical computer-based systems** are systems that include hardware and software components but not procedures and processes.

Examples of technical systems include televisions, mobile phones, and other equipment with embedded software. Applications for PCs, computer games, and mobile devices are also technical systems. Individuals and organizations use technical systems for a particular purpose, but knowledge of this purpose is not part of the technical system. For example, the word processor that we are using (Microsoft Word) is not aware that it is being used to write a book.

2. **Sociotechnical systems** include one or more technical systems but, crucially, also people, who understand the purpose of the system, within the system itself. Sociotechnical systems have defined operational processes, and people (the operators) are essential parts of the system. They are governed by organizational policies and rules and may be affected by external constraints such as national laws and regulatory policies. For example, this book was created through a sociotechnical publishing system that includes various processes (creation, editing, layout, etc.) and technical systems (Microsoft Word and Excel, Adobe Illustrator, Indesign, etc.).

Four Overlapping Stages in the Lifetime of Large, Complex Systems (Sommerville, n.d.):

1. **Conceptual Design.** This initial system engineering activity develops the concept of the type of system that is required. It sets out, in nontechnical language, the purpose of the system, why it is needed, and the high-level features that users might expect to see in the system. It may also describe broad constraints, such as the need for interoperability with other systems. These limit the freedom of systems engineers in designing and developing the system.

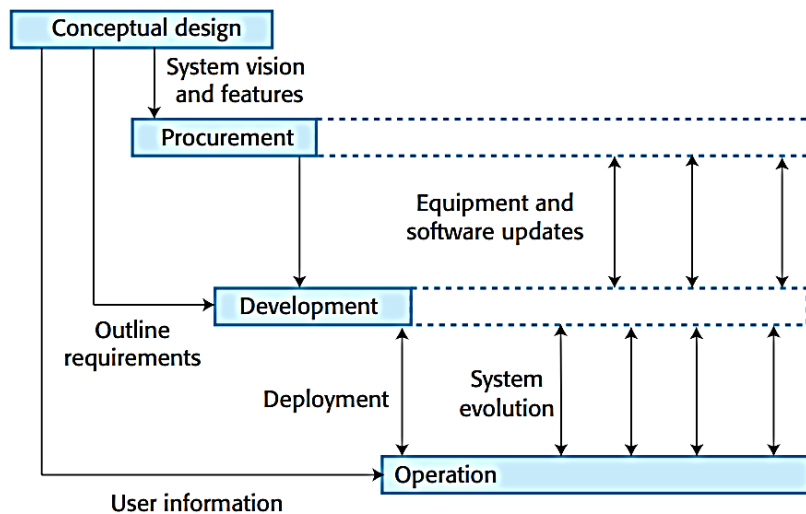


Figure 5.1 Stages of Systems Engineering

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

2. **Procurement or Acquisition.** During this stage, the conceptual design is further developed so that information is available to make decisions about the contract for the system development. This may involve making decisions about the distribution of functionality across hardware, software, and operational processes. You also make decisions about which hardware and software has to be acquired, which suppliers should develop the system, and the terms and conditions of the supply contract.
3. **Development.** During this stage, the system is developed. Development processes include requirements definition, system design, hardware and software engineering, system integration, and testing. Operational processes are defined, and the training courses for system users are designed.
4. **Operation.** At this stage, the system is deployed, users are trained, and the system is brought into use. The planned operational processes usually then have to change to reflect the real working environment where the system is used. Over time, the system evolves as new requirements are identified. Eventually, the system declines in value, and it is decommissioned and replaced.

Figure 5.1 shows the interactions between these stages. The conceptual design activity is a basis for the system procurement and development but is also used to provide information to users about the system. Development and procurement overlap and further

procurement during development, and operation may be needed as new equipment and software become available. Once the system is operational, requirements changes are inevitable; implementing these changes requires further development and, perhaps, software and hardware procurement (Sommerville, n.d.).

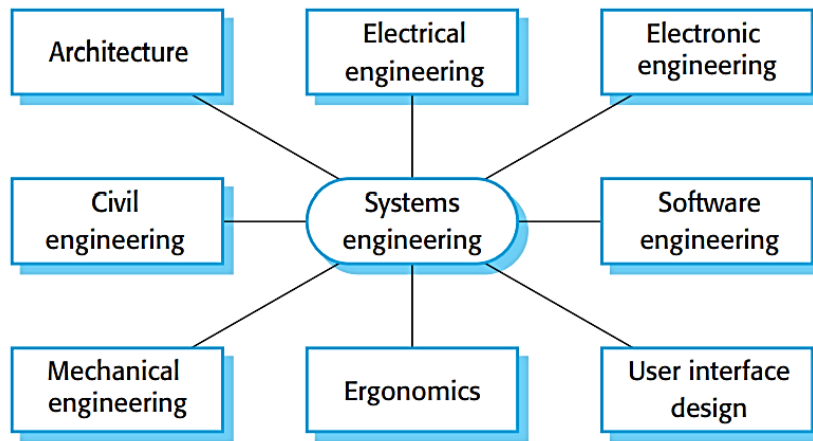


Figure 5.2 Professional Disciplines Involved in ATC Systems Engineering

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Figure 5.2 illustrates the technical disciplines that may be involved in the procurement and development of a new system for air traffic management (Sommerville, n.d.).

- Architects and civil engineers are involved because new air traffic management systems usually have to be installed in a new building.
- Electrical and mechanical engineers are involved to specify and maintain the power and air conditioning.
- Electronic engineers are concerned with computers, radars, and other equipment.
- Ergonomists design the controller workstations and software engineers, and user interface designers are responsible for the software in the system.

The involvement of a range of professional disciplines is essential because of the different types of components in complex systems. However, differences and misunderstandings between disciplines can lead to inappropriate design decisions. These poor decisions can delay the system's development or make it less suitable for its intended purpose.

There are three reasons why there may be misunderstandings or other differences between engineers with different backgrounds (Sommerville, n.d.):

1. Different professional disciplines often use the same words, but these words do not always mean the same thing. That is why misunderstandings are common in discussions between engineers from different backgrounds. If these are not discovered and resolved during system development, they can lead to errors in delivered systems. For example, an electronic engineer may know a bit about C programming but may not understand that a method in Java is like a function in C.
2. Each discipline makes assumptions about what other disciplines can or cannot do. These assumptions are often based on an inadequate understanding of what is possible. For example, an electronic engineer may decide that all signal processing (a computationally intensive task) should be done by software to simplify the hardware design. However, this may mean significantly greater software effort to ensure that the system processor can cope with the amount of computation that is resolved.
3. Disciplines try to protect their professional boundaries and may argue for certain design decisions because these decisions will call for their professional expertise. Therefore, a software engineer may argue for a software-based door locking system in a building, although a mechanical, key-based system may be more reliable.

Lesson 2. Sociotechnical Systems

Large-scale systems are, with a few exceptions like sociotechnical systems. That is, they do not just include software and hardware but also people, processes, and organizational policies. Sociotechnical systems are enterprise systems that are intended to help deliver a business purpose. This purpose might be to increase sales, reduce material used in manufacturing, collect taxes, maintain a safe airspace, and so on.

Because they are embedded in an organizational environment, the procurement, development, and use of these systems are influenced by the organization's policies and procedures, as well as by its working culture. The users of the system are people who are influenced by the way the organization is managed and by their interactions with other people inside and outside of the organization (Sommerville, n.d.).

Different people within the organization will see the boundaries of the system in different ways. This is significant because establishing what is and what is not in the scope of

the system is important when defining the system requirements. Figure 5.3 below illustrates this problem. The diagram shows a sociotechnical system as a set of layers, where each layer contributes, in some way, to the functioning of the system. At the core is a software-intensive technical system and its operational processes (shaded in Figure 5.3).

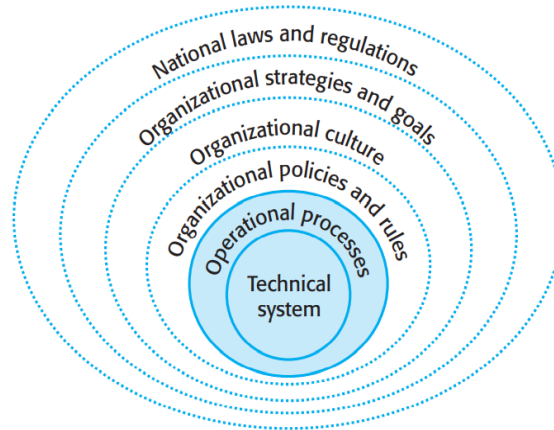


Figure 5.3 Layered Structure of Sociocultural System

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Generally, large sociotechnical systems are used in organizations. When you are designing and developing sociotechnical systems, you need to understand, as far as possible, the organizational environment in which they will be used. If you don't, the systems may not meet business needs. Users and their managers may reject the system or fail to use it to its full potential.

Figure 5.4 shows the key elements in an organization that may affect the requirements, design, and operation of a sociotechnical system. A new system may lead to changes in some or all of these elements (Sommerville, n.d.):

Organizational Elements (Sommerville, n.d.)

1. **Process Changes.** A new system may mean that people have to change the way that they work. If so, training will certainly be required. If changes are significant, or if they involve people losing their jobs, there is a danger that the users will resist the introduction of the system.
2. **Job Changes.** New systems may deskill the users in an environment or cause them to change the way they work. If so, users may actively resist the introduction of the system into the organization. Professional staff, such as doctors or teachers, may resist system

designs that require them to change their normal way of working. The people involved may feel that their professional expertise is being eroded and that their status in the organization is being reduced by the system.

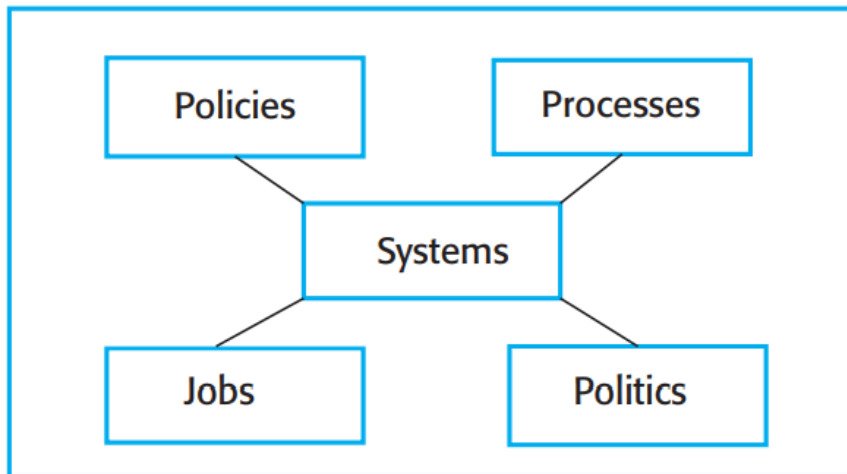


Figure 5.4 Organizational Elements

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

3. **Organizational Policies.** The proposed system may not be completely consistent with organizational policies (e.g., on privacy). This may require system changes, policy changes, or process changes to bring the system and policies into line.
4. **Organizational Politics.** The system may change the political power structure in an organization. For example, if an organization is dependent on a complex system, those who control access to that system have a great deal of political power. Alternatively, if an organization reorganizes itself into a different structure, this may affect the requirements and use of the system.

Sociotechnical systems are complex systems, which means that it is practically impossible to have a complete understanding, in advance, of their behavior. This complexity leads to three important characteristics of sociotechnical systems (Sommerville, n.d.):

1. They have emergent properties that are properties of the system as a whole, rather than associated with individual parts of the system. Emergent properties depend on both the system components and the relationships between them. Some of these relationships only come into existence when the system is integrated from its components, so the emergent

properties can only be evaluated at that time. Security and dependability are examples of important emergent system properties.

2. They are nondeterministic, so that when presented with a specific input, they may not always produce the same output. The system's behavior depends on the human operators, and people do not always react in the same way. Furthermore, use of the system may create new relationships between the system components and hence change its emergent behavior.
3. The system's success criteria are subjective rather than objective. The extent to which the system supports organizational objectives does not just depend on the system itself. It also depends on the stability of these objectives, the relationships and conflicts between organizational objectives, and how people in the organization interpret these objectives. New management may reinterpret the organizational objectives that a system was designed to support so that a "successful" system may then be seen as no longer fit for its intended purpose.

Sociotechnical considerations are often critical in determining whether or not a system has successfully met its objectives. Unfortunately, taking these into account is very difficult for engineers who have little experience of social or cultural studies. To help understand the effects of systems on organizations, various sociotechnical systems methodologies have been proposed. My paper on sociotechnical systems design discusses the advantages and disadvantages of these sociotechnical design methodologies (Baxter and Sommerville, 2011).

Emergent Properties

According to Checkland (1981) emergent properties cannot be attributed to any specific part of the system. Rather, they only emerge once the system components have been integrated. Some emergent properties, such as weight, can be derived directly from the subsystem properties. More often, however, they emerge from a combination of subsystem properties and subsystem relationships. The system property cannot be calculated directly from the properties of the individual system components. Examples of emergent properties are shown in Figure 5.5.

Property	Description
Reliability	System reliability depends on component reliability, but unexpected interactions can cause new types of failure and therefore affect the reliability of the system.
Repairability	This property reflects how easy it is to fix a problem with the system once it has been discovered. It depends on being able to diagnose the problem, access the components that are faulty, and modify or replace these components.
Security	The security of the system (its ability to resist attack) is a complex property that cannot be easily measured. Attacks may be devised that were not anticipated by the system designers and so may defeat built-in safeguards.
Usability	This property reflects how easy it is to use the system. It depends on the technical system components, its operators, and its operating environment.
Volume	The volume of a system (the total space occupied) depends on how the component assemblies are arranged and connected.

Figure 5.5 Examples of Emergent Properties

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Two Types of Emergent Properties (Sommerville, n.d.)

1. Functional emergent properties, when the purpose of a system only emerges after its components are integrated. For example, a bicycle has the functional property of being a transportation device once it has been assembled from its components.
2. Non-functional emergent properties, which relate to the behavior of the system in its operational environment. Reliability, performance, safety, and security are examples of these properties. These system characteristics are critical for computer-based systems, as failure to achieve a minimum defined level in these properties usually makes the system unusable. Some users may not need some of the system functions, so the system may be acceptable without them. However, a system that is unreliable or too slow is likely to be rejected by all its users

Emergent properties, such as reliability, depend on both the properties of individual components and their interactions or relationships. For example, the reliability of a sociotechnical system is influenced by three things (Sommerville, n.d.):

1. **Hardware Reliability.** What is the probability of hardware components failing, and how long does it take to repair a failed component?
2. **Software Reliability.** How likely is it that a software component will produce an incorrect output? Software failure is unlike hardware failure in that software does not wear out.

Failures are often transient. The system carries on working after an incorrect result has been produced.

3. **Operator Reliability.** How likely is it that the operator of a system will make an error and provide an incorrect input? How likely is it that the software will fail to detect this error and propagate the mistake?

Hardware, software, and operator reliability are not independent but affect each other in unpredictable ways. Let's say a hardware component in a system starts to go wrong. Hardware failure can sometimes generate spurious signals that are outside the range of inputs expected by the software. The software can then behave unpredictably and produce unexpected outputs. These may confuse and consequently cause stress in the system operator (Sommerville, n.d.).

Non-Determinism

A deterministic system is one that is absolutely predictable. If we ignore issues of concurrency, software systems that run on reliable hardware are deterministic. When they are presented with a sequence of inputs, they will always produce the same sequence of outputs. Of course, there is no such thing as completely reliable hardware, but hardware is usually reliable enough to think of hardware systems as deterministic (Sommerville, n.d.).

People, on the other hand, are non-deterministic. When presented with exactly the same input (say a request to complete a task), their responses will depend on their emotional and physical state, the person making the request, other people in the environment, and whatever else they are doing. Sometimes they will be happy to do the work, and, at other times, they will refuse; sometimes they will perform a task well, and sometimes they will do it badly.

Sociotechnical systems are nondeterministic partly because they include people and partly because changes to the hardware, software, and data in these systems are so frequent. The interactions between these changes are complex, and so the behavior of the system is unpredictable. Users do not know when and why changes have been made, so they see the system as nondeterministic.

Non-determinism is often seen as a bad thing, and it is felt that designers should try to avoid nondeterministic behavior wherever possible. In fact, in sociotechnical systems, non-determinism has important benefits. It means that the behavior of a system is not fixed for all

time but can change depending on the system's environment. For example, operators may observe that a system is showing signs of failure. Instead of using the system normally, they can change their behavior to diagnose and recover from the detected problems (Sommerville, n.d.).

Success Criteria

This makes it difficult to define the success criteria for a system. How do you decide if a new system contributes to the business goals of the company that paid for the system? The judgment of success is not usually made against the original reasons for procuring and developing the system. Rather, it is based on whether or not the system is effective at the time it is deployed.

As the business environment can change very quickly, the business goals may have changed significantly during the development of the system. The situation is even more complex when there are multiple conflicting goals that are interpreted differently by different stakeholders. For instance, the system on which the mental care system is based was designed to support two separate business goals (Sommerville, n.d.):

1. To improve the quality of care for sufferers from mental illness.
2. To improve the cost-effectiveness of treatments by providing managers with detailed reports of care provided and the costs of that care.

Any success criteria that are established in the early stages of the systems engineering process have to be regularly reconsidered during system development and use. You cannot evaluate these criteria objectively as they depend on the system's effect on its environment and its users. A system may apparently meet its requirements as originally specified but be practically useless because of changes in the environment where it is used (Sommerville, n.d.).

Lesson 3. Conceptual Design

Once an idea for a system has been suggested, conceptual design is the very first thing that you do in the systems engineering process. In the conceptual design phase, you take that initial idea, investigate its feasibility, and develop it to create an overall vision of a

system that could be developed. You then have to describe the envisaged system so that nonexperts, such as system users, senior company decision makers, or politicians, can understand what you are proposing.

There is an obvious overlap between conceptual design and requirements engineering. As part of the conceptual design process, you have to imagine how the proposed system will be used. This may involve discussions with potential users and other stakeholders, focus groups, and observations of how existing systems are used. The goal of these activities is to understand how users work, what is important to them, and what practical constraints on the system there might be (Sommerville, n.d.).

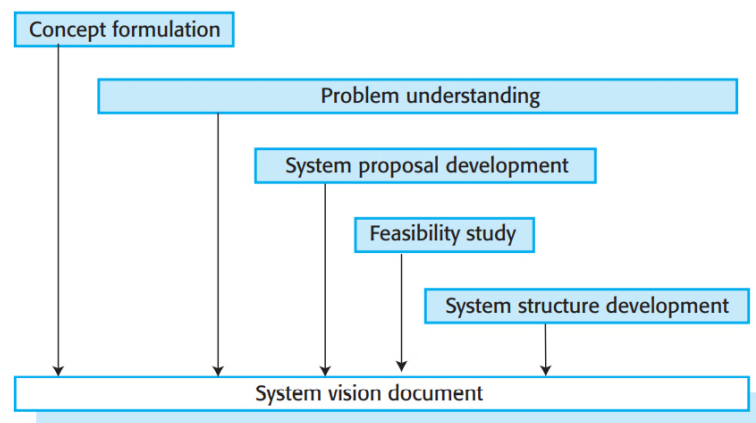


Figure 5.6 Conceptual Design Activities

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Figure 5.6 shows activities that may be part of the conceptual design process. Conceptual design should always be a team process that involves people from different backgrounds. As an example, in a digital learning environment, the design team included teachers, education researchers, software engineers, system administrators, and system managers (Sommerville, n.d.).

Concept formulation is the first stage of the process where we try to refine an initial statement of needs and work out what type of system would be best to meet the needs of system stakeholders.

Initially, we were tasked with proposing an intranet for information sharing across schools that was easier to use than the current system. However, after discussions with teachers, we discovered that this was not really what was required. The existing system was awkward to use, but people had found workarounds. What was really required was a flexible

digital learning environment that could be adapted by adding subject and age-specific tools and content that are freely available on the Internet.

Tips on How to Understand a Problem (Sommerville, n.d.)

1. Discuss with users and other stakeholders how they do their work;
2. Find out what is important to them, what are the barriers that stop them from doing what they want to do, and their ideas of what changes are required;
3. Be open-minded (it is their problem, not yours); and
4. Be prepared to change your ideas when the reality does not match your initial vision.

In the system proposal development stage, the conceptual design team set out their ideas for alternative systems and these are the basis for a feasibility study to decide which of the ideas are worth further development.

In a feasibility study, you should look at comparable systems that have been developed elsewhere and technological issues (e.g., use of mobile devices) that may affect use of the system. Then you need to assess whether or not the system could be implemented using current hardware and software technologies (Sommerville, n.d.).

Lesson 4. System Procurement

System procurement or system acquisition is a process whose outcome is a decision to buy one or more systems from system suppliers. At this stage, decisions are made on the scope of a system that is to be purchased, system budgets and timescales, and high-level system requirements. Using this information, further decisions are then made on whether to procure a system, the type of system required, and the supplier or suppliers of the system. The drivers for these decisions are (Sommerville, n.d.):

1. The replacement of other organizational systems If the organization has a mixture of systems that cannot work together or that are expensive to maintain, then procuring a replacement system, with additional capabilities, may lead to significant business benefits.
2. The need to comply with external regulations Increasingly, businesses are regulated and have to demonstrate compliance with externally defined regulations (e.g., Sarbanes–Oxley accounting regulations in the United States). Compliance may require the

replacement of noncompliant systems or the provision of new systems specifically to monitor compliance.

3. External competition If a business needs to compete more effectively or maintain a competitive position, managers may decide to buy new systems to improve business efficiency or effectiveness. For military systems, the need to improve capability in the face of new threats is an important reason for procuring new systems.
4. Business reorganization Businesses and other organizations frequently restructure with the intention of improving efficiency and/or customer service. Reorganizations lead to changes in business processes that require new systems support.
5. Available budget. The budget that is available is an obvious factor in determining the scope of new systems that can be procure

In addition, new government systems are often procured to reflect political changes and political policies. For example, politicians may decide to buy new surveillance systems, which they claim will counter terrorism. Buying such systems shows voters that they are taking action.

Large complex systems are usually engineered using a mixture of off-the-shelf and specially built components. They are often integrated with existing legacy systems and organizational databases. When legacy systems and off-the-shelf systems are used, new custom software may be needed to integrate these components. The new software manages the component interfaces so that these components can interoperate. The need to develop this “glueware” is one reason why the savings from using off-the-shelf components are sometimes not as great as anticipated (Sommerville, n.d.).

Three types of systems or system components may have to be procured (Sommerville, n.d.):

1. Off-the-shelf applications that may be used without change and that need only minimal configuration for use.
2. Configurable application or ERP systems that have to be modified or adapted for use either by modifying the code or by using inbuilt configuration features, such as process definitions and rules.
3. Custom systems that have to be specially designed and implemented for use.

Each of these components tends to follow a different procurement process. Figure 1.7 illustrates the main features of the procurement process for these types of system.

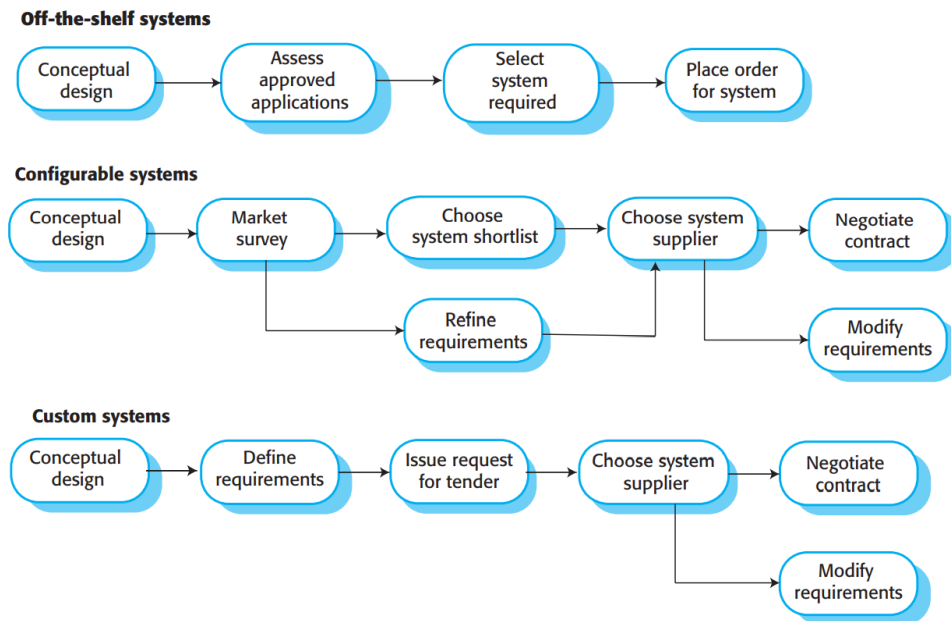


Figure 5.7 Systems Procurement Processes

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

According to Sommerville (2016) the key issues that affect procurement processes are:

1. **Organizations often have an approved and recommended set of application software that has been checked by the IT department.** It is usually possible to buy or acquire open-source software from this set directly without the need for detailed justification. For example, in the iLearn system, we recommended that Wordpress should be made available for student and staff blogs. If microphones are needed, off-the-shelf hardware can be bought. There are no detailed requirements, and the users adapt to the features of the chosen application.
2. **Off-the-shelf components do not usually match requirements exactly, unless the requirements have been written with these components in mind.** Therefore, choosing a system means that you have to find the closest match between the system requirements and the facilities offered by off-the-shelf systems. ERP and other large-scale application systems usually fall into this category. You may then have to modify the requirements to fit in with the system assumptions. This can have knock-on effects on other subsystems. You also usually have an extensive configuration process to tailor and adapt the application or ERP system to the buyer's working environment.
3. **When a system is to be built specially, the specification of requirements is part of the contract for the system being acquired.** It is therefore a legal as well as a technical

document. The requirements document is critical, and procurement processes of this type usually take a considerable amount of time.

4. **For public sector systems in particular, there are detailed rules and regulations that affect the procurement of systems.** For example, in the European Union, all public sector systems over a certain price must be open to tender by any supplier in Europe. This requires detailed tender documents to be drawn up and the tender to be advertised across Europe for a fixed period of time. Not only does this rule slow down the procurement process, it also tends to inhibit agile development. It forces the system buyer to develop requirements so that all companies have enough information to bid for the system contract.
5. **For application systems that require change or for custom systems, there is usually a contract negotiation period when the customer and supplier negotiate the terms and conditions for development of the system.** Once a system has been selected, you may negotiate with the supplier on costs, license conditions, possible changes to the system, and other contractual issues. For custom systems, negotiations are likely to involve payment schedules, reporting, acceptance criteria, requirements change requests, and costs of system changes. During this process, requirements changes may be agreed that will reduce the overall costs and avoid some development problems.

Complex sociotechnical systems are rarely developed “in house” by the buyer of the system. Rather, external systems companies are invited to bid for the systems engineering contract. The customer’s business is not systems engineering, so its employees do not have the skills needed to develop the systems themselves (Sommerville, n.d.).

For complex hardware/software systems, it may be necessary to use a group of suppliers, each with a different type of expertise. For large systems, such as an air traffic management system, a group of suppliers may form a consortium to bid for a contract. The consortium should include all of the capabilities required for this type of system. For an ATC system, this would include computer hardware suppliers, software companies, peripheral suppliers, and suppliers of specialist equipment such as radar systems (Sommerville, n.d.).

Customers do not usually wish to negotiate with multiple suppliers, so the contract is usually awarded to a principal contractor, who coordinates the project. The principal contractor coordinates the development of different subsystems by subcontractors. The subcontractors design and build parts of the system to a specification that is negotiated with the principal

contractor and the customer. Once completed, the principal contractor integrates these components and delivers them to the customer.

Decisions made at the procurement stage of the systems engineering process are critical for later stages in that process. Poor procurement decisions often lead to problems such as late delivery of a system and development of systems that are unsuited to their operational environment. If the wrong system or the wrong supplier is chosen, then the technical processes of system and software engineering become more complex.

Many bad procurement decisions stem from political rather than technical causes. Senior management may wish to have more control and so demand that a single system is used across an organization. Suppliers may be chosen because they have a longstanding relationship with a company rather than because they offer the best technology. Managers may wish to maintain compatibility with existing systems because they feel threatened by new technologies (Sommerville, n.d.).

Lesson 5. System Development

System development is a complex process in which the elements that are part of the system are developed or purchased and then integrated to create the final system. The system requirements are the bridge between the conceptual design and the development processes (Sommerville, n.d.).

During conceptual design, business and high-level functional and non-functional system requirements are defined. Think of this as the start of development, hence the overlapping processes. Once contracts for the system elements have been agreed, more detailed requirements engineering takes place. Figure 1.8 is a model of the systems development process. Systems engineering processes usually follow a “waterfall” process model. Although the waterfall model is inappropriate for most types of software development, higher-level systems engineering processes are plan-driven processes that still follow this model (Sommerville, n.d.).

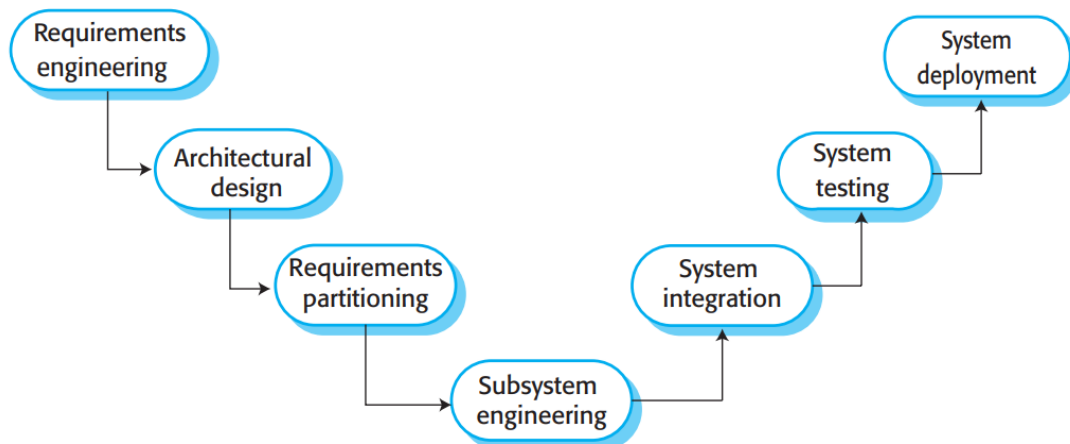


Figure 5.8 The Systems Development Processes

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Plan-driven processes are used in systems engineering because different elements of the system are independently developed. Different contractors are working concurrently on separate subsystems. Therefore, the interfaces to these elements have to be designed before development begins. For systems that include hardware and other equipment, changes during development can be very expensive or, sometimes, practically impossible (Sommerville, n.d.).

It is essential therefore, that the system requirements are fully understood before hardware development or building work begins. One of the most confusing aspects of systems engineering is that companies use different terminology for each stage of the process.

Sometimes, requirements engineering is part of the development process, and sometimes it is a separate activity. However, after conceptual design, there are seven fundamental development activities (Sommerville, n.d.):

1. Requirements engineering is the process of refining, analyzing, and documenting the high-level and business requirements identified in the conceptual design.
2. Architectural design overlaps significantly with the requirements engineering process. The process involves establishing the overall architecture of the system identifying the different system components, and understanding the relationships between them.
3. Requirements partitioning is concerned with deciding which subsystems (identified in the system architecture) are responsible for implementing the system requirements. Requirements may have to be allocated to hardware, software, or operational processes and prioritized for implementation. Ideally, you should allocate requirements to individual subsystems so that the implementation of a critical requirement does not need subsystem

collaboration. However, this is not always possible. At this stage you also decide on the operational processes and on how these are used in the requirements implementation.

4. Subsystem engineering involves developing the software components of the system, configuring off-the-shelf hardware and software, designing, if necessary, special-purpose hardware, defining the operational processes for the system, and re-designing essential business processes.
5. System integration is the process of putting together system elements to create a new system. Only then do the emergent system properties become apparent.
6. System testing is an extended activity where the whole system is tested and problems are exposed. The subsystem engineering and system integration phases are reentered to repair these problems, tune the performance of the system, and implement new requirements. System testing may involve both testing by the system developer and acceptance/user testing by the organization that has procured the system.
7. System deployment is the process of making the system available to its users, transferring data from existing systems, and establishing communications with other systems in the environment. The process culminates with a “go live,” after which users start to use the system to support their work.

Although the overall process is plan-driven, the processes of requirements development and system design are inextricably linked. The requirements and the high-level design are developed concurrently. Constraints posed by existing systems may limit design choices, and these choices may be specified in the requirements. You may have to do some initial design to structure and organize the requirements engineering process. As the design process continues, you may discover problems with existing requirements and new requirements may emerge. Consequently, you can think of these linked processes as a spiral, as shown in Figure 5.9.

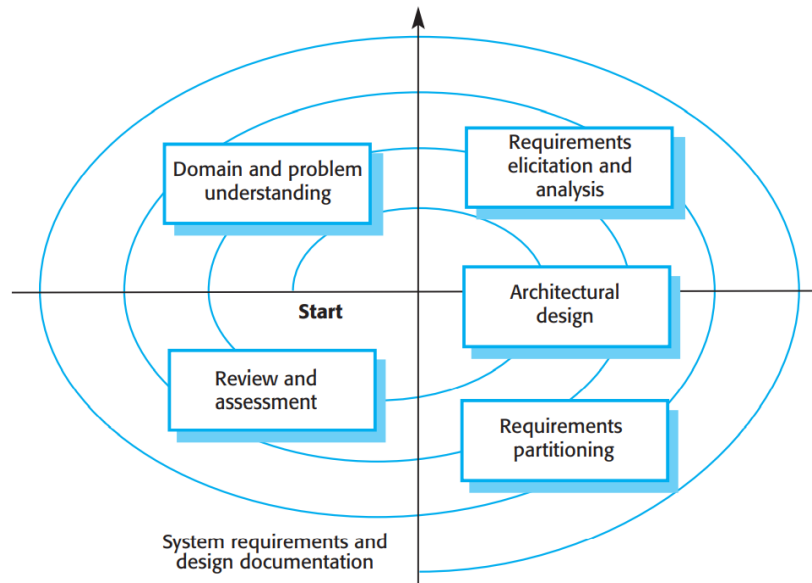


Figure 5.9 Requirements and Design Spiral

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

The spiral reflects the reality that requirements affect design decisions and vice versa, and so it makes sense to interleave these processes. Starting in the center, each round of the spiral may add detail to the requirements and the design (Sommerville, n.d.).

As subsystems are identified in the architecture, decisions are made on the responsibilities of these subsystems for providing the system requirements. Some rounds of the spiral may focus on requirements, others on design. Sometimes new knowledge collected during the requirements and design process means that the problem statement itself has to be changed.

For almost all systems, many possible designs meet the requirements. These cover a range of solutions that combine hardware, software, and human operations. The solution that you choose for further development may be the most appropriate technical solution that meets the requirements.

However, wider organizational and political considerations may influence the choice of solution. For example, a government client may prefer to use national rather than foreign suppliers for its system, even if national products are technically inferior. These influences usually take effect in the review and assessment phase of the spiral model where designs and requirements may be accepted or rejected. The process ends when a review decides that the requirements and high-level design are sufficiently detailed for subsystems to be specified and designed (Sommerville, n.d.).

Subsystem engineering involves designing and building the system's hardware and software components. For some types of systems, such as spacecraft, all hardware and software components may be designed and built during the development process. However, in most systems, some components are bought rather than developed (Sommerville, n.d.).

It is usually much cheaper to buy existing products than to develop special-purpose components. However, if you buy large off-the-shelf systems, such as ERP systems, there is a significant cost in configuring these systems for use in their operational environment. Subsystems are usually developed in parallel. When problems that cut across subsystem boundaries are encountered, a system modification request must be made. Where systems involve extensive hardware engineering, making modifications after manufacturing has started is usually very expensive.

Often "workarounds" that compensate for the problem must be found. These workarounds usually involve software changes to implement new requirements. During systems integration, you take the independently developed subsystems and put them together to make up a complete system. This integration can be achieved using a "big bang" approach, where all the subsystems are integrated at the same time. However, for technical and managerial reasons, an incremental integration process where subsystems are integrated one at a time is the best approach (Sommerville, n.d.):

1. It is usually impossible to schedule the development of all the subsystems so that they are all finished at the same time.
2. Incremental integration reduces the cost of error location. If many subsystems are simultaneously integrated, an error that arises during testing may be in any of these subsystems. When a single subsystem is integrated with an already working system, errors that occur are probably in the newly integrated subsystem or in the interactions between the existing subsystems and the new subsystem.

As an increasing number of systems are built by integrating off-the-shelf hardware and software application systems, the distinction between implementation and integration is becoming blurred. In some cases, there is no need to develop new hardware or software. Essentially, systems integration is the implementation phase of the system (Sommerville, n.d.).

During and after the integration process, the system is tested. This testing should focus on testing the interfaces between components and the behavior of the system as a whole. Inevitably, testing also reveals problems with individual subsystems that have to be repaired.

Testing takes a long time, and a common problem in system development is that the testing team may run out of either budget or time. This problem can lead to the delivery of error-prone systems that need be repaired after they have been deployed.

Subsystem faults that are a consequence of invalid assumptions about other subsystems are often exposed during system integration. This may lead to disputes between the contractors responsible for implementing different subsystems. When problems are discovered in subsystem interaction, the contractors may argue about which subsystem is faulty. Negotiations on how to solve the problems can take weeks or months.

The final stage of the system development process is system delivery and deployment. The software is installed on the hardware and is readied for operation. This may involve more system configuration to reflect the local environment where it is used, the transfer of data from existing systems, and the preparation of user documentation and training. At this stage, you may also have to reconfigure other systems in the environment to ensure that the new system interoperates with them (Sommerville, n.d.).

Although system deployment is straightforward in principle, it is often more difficult than anticipated. The user environment may be different from that anticipated by the system developers. Adapting the system to make it work in an unexpected environment can be difficult. The existing system data may require extensive clean-up, and parts of it may involve more effort than expected. The interfaces to other systems may not be properly documented.

You may find that the planned operational processes have to be changed because they are not compatible with the operational processes for other systems. User training is often difficult to arrange, with the consequence that, initially at least, users are unable to access the capabilities of the system. System deployment can therefore take much longer and cost much more than anticipated (Sommerville, n.d.).

Lesson 6. System Operation and Evolution

Operational processes are the processes that are involved in using the system as intended by its designers. For example, operators of an air traffic control system follow specific processes when aircraft enter and leave airspace, when they have to change height or speed,

when an emergency occurs, and so on. For new systems, these operational processes have to be defined and documented during the system development process.

Operators may have to be trained and other work processes adapted to make effective use of the new system. Undetected problems may arise at this stage because the system specification may contain errors or omissions. While the system may perform to specification, its functions may not meet the real operational needs. Consequently, the operators may not use the system as its designers intended.

While user interface design is important, studies have shown that, given time, users can adapt to complex interfaces. As they become experienced, they prefer ways of using the system quickly rather than easily. This means that when designing systems, you should not simply cater for inexperienced users but you should design the user interface to be adaptable for experienced users.

Some people think that system operators are a source of problems in a system and that we should move toward automated systems where operator involvement is minimized. In my opinion, there are two problems with this approach (Sommerville, n.d.):

1. It is likely to increase the technical complexity of the system because it has to be designed to cope with all anticipated failure modes. This increases the costs and time required to build the system. Provision also has to be made to bring in people to deal with unanticipated failures.
2. People are adaptable and can cope with problems and unexpected situations. Thus, you do not have to anticipate everything that could possibly go wrong when you are specifying and designing the system.

People have a unique capability of being able to respond effectively to the unexpected, even when they have never had direct experience of these unexpected events or system states. Therefore, when things go wrong, the system operators can often recover the situation by finding workarounds and using the system in nonstandard ways. Operators also use their local knowledge to adapt and improve processes. Normally, the actual operational processes are different from those anticipated by the system designers (Sommerville, n.d.).

Consequently, you should design operational processes to be flexible and adaptable. The operational processes should not be too constraining; they should not require operations to be done in a particular order; and the system software should not rely on a specific process

being followed. Operators usually improve the process because they know what does and does not work in a real situation.

A problem that may only emerge after the system goes into operation is the operation of the new system alongside existing systems. There may be physical problems of incompatibility, or it may be difficult to transfer data from one system to another. More subtle problems might arise because different systems have different user interfaces. Introducing a new system may increase the operator error rate, as the operators use user interface commands for the wrong system (Sommerville, n.d.).

System Evolution

As discussed by Sommerville (2016) large, complex systems usually have a long lifetime. Complex hardware/software systems may remain in use for more than 20 years, even though both the original hardware and software technologies used are obsolete. There are several reasons for this longevity, as shown in Figure 5.10.

Factor	Rationale
Investment cost	The costs of a systems engineering project may be tens or even hundreds of millions of dollars. These costs can only be justified if the system can deliver value to an organization for many years.
Loss of expertise	As businesses change and restructure to focus on their core activities, they often lose engineering expertise. This may mean that they lack the ability to specify the requirements for a new system.
Replacement cost	The cost of replacing a large system is very high. Replacing an existing system can be justified only if this leads to significant cost savings over the existing system.
Return on investment	If a fixed budget is available for systems engineering, spending on new systems in some other area of the business may lead to a higher return on investment than replacing an existing system.
Risks of change	Systems are an inherent part of business operations, and the risks of replacing existing systems with new systems cannot be justified. The danger with a new system is that things can go wrong in the hardware, software, and operational processes. The potential costs of these problems for the business may be so high that they cannot take the risk of system replacement.
System dependencies	Systems are interdependent and replacing one of these systems may lead to extensive changes in other systems.

Figure 5.10 Factors That Influence Systems Lifetimes

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Over their lifetime, large complex systems change and evolve to correct errors in the original system requirements and to implement new requirements that have emerged. The

system's computers are likely to be replaced with new, faster machines. The organization that uses the system may reorganize itself and hence use the system in a different way. The external environment of the system may change, forcing changes to the system. Hence, evolution is a process that runs alongside normal system operational processes.

System evolution involves reentering the development process to make changes and extensions to the system's hardware, software, and operational processes. System evolution, like software evolution, is inherently costly for several reasons (Sommerville, n.d.):

1. Proposed changes have to be analyzed very carefully from a business and a technical perspective. Changes have to contribute to the goals of the system and should not simply be technically motivated.
2. Because subsystems are never completely independent, changes to one subsystem may have side-effects that adversely affect the performance or behavior of other subsystems. Consequent changes to these subsystems may therefore be needed.
3. The reasons for original design decisions are often unrecorded. Those responsible for the system evolution have to work out why particular design decisions were made.
4. As systems age, their structure becomes corrupted by change, so the costs of making further changes increases.

Systems that have been in use for many years are often reliant on obsolete hardware and software technology. These "legacy systems" are sociotechnical computer-based systems that have been developed using technology that is now obsolete. However, they don't just include legacy hardware and software. They also rely on legacy processes and procedures old ways of doing things that are difficult to change because they rely on legacy software (Sommerville, n.d.).

Changes to one part of the system inevitably involve changes to other components. Changes made to a system during system evolution are often a source of problems and vulnerabilities. If the people implementing the changes are different from those who developed the system, they may be unaware that a design decision was taken for dependability and security reasons. Therefore, they may change the system and lose some safeguards that were deliberately implemented when the system was built.

Furthermore, as testing is so expensive, complete retesting may be impossible after every system change. Consequently, testing may not discover the adverse side effects of changes that introduce or expose faults in other system components (Sommerville, n.d.)

MODULE 6

SYSTEM OF SYSTEMS



Introduction

We need software engineering because we create large and complex software systems. The discipline emerged in the 1960s because the first attempts to build large software systems mostly went wrong. Creating software was much more expensive than expected, took longer than planned, and the software itself was often unreliable. To address these problems, we have developed a range of software engineering techniques and technologies, which have been remarkably successful. We can now build systems that are much larger, more complex, much more reliable, and more effective than the software systems of the 1970s (Sommerville, n.d.).

The increase in size of software systems since the introduction of software engineering has been remarkable. Today's large systems may be a hundred or even a thousand times larger than the "large" systems of the 1960s. Northrop and her colleagues (Northrop et al. 2006) suggested in 2006 that we would shortly see the development of systems with a billion lines of code. Almost 10 years after this prediction, I suspect such systems are already in use.

What do we mean when we talk about a system of systems? As Hitchens says (Hitchens 2009), from a general systems perspective, there is no difference between a system and a system of systems. Both have emergent properties and can be composed from subsystems. However, from a software engineering perspective, there is a useful distinction between these terms. This distinction is sociotechnical rather than technical:

A system of systems is a system that contains two or more independently managed elements.

This means that there is no single manager for all of the parts of the system of systems and that different parts of a system are subject to different management and control policies and rules. As we shall see, distributed management and control has a profound effect on the overall complexity of the system.

This definition of systems of systems says nothing about the size of systems of systems. A relatively small system that includes services from different providers is a system of systems. Some of the problems of SoS engineering apply to such small systems, but the real challenges emerge when the constituent systems are themselves large-scale systems.

The focus here on systems of systems where the system elements are software systems rather than hardware such as aircraft, military vehicles, or radars. Systems of software systems are created by integrating separate software systems, and, at the time of writing, most software SoS include a relatively small number of separate systems.

Each constituent system is usually a complex system in its own right. However, it is predicted that, over the next few years, the size of software SoS is likely to grow significantly as more and more systems are integrated to make use of the capabilities that they offer.

Examples of systems of systems of software systems are (Sommerville, n.d.):

1. A cloud management system that handles local private cloud management and management of servers on public clouds such as Amazon and Microsoft.
2. An online banking system that handles loan requests and that connects to a credit reference system provided by credit reference agencies to check the credit of applicants.
3. An emergency information system that integrates information from police, ambulance, fire, and coast guard services about the assets available to deal with civil emergencies such as flooding and large-scale accidents.
4. The digital learning environment (iLearn) as an example is a system which provides a range of learning support by integrating separate software systems such as Microsoft Office 365, virtual learning environments such as Moodle, simulation modeling tools, and content such as newspaper archives.

Maier (1998) identified five essential characteristics of system of systems:

1. Operational independence of elements Parts of the system are not simply components but can operate as useful systems in their own right. The systems within the SoS evolve independently.
2. Managerial independence of elements Parts of the system are “owned” and managed by different organizations or by different parts of a larger organization. Therefore, different rules and policies apply to the management and evolution of these systems.

3. Evolutionary development SoS are not developed in a single project but evolve over time from their constituent systems.
4. Emergence SoS have emergent characteristics that only become apparent after the SoS has been created. Emergence is a characteristic of all systems, but it is particularly important in SoS.
5. Geographical distribution of elements the elements of a SoS are often geographically distributed across different organizations. This is important technically because it means that an externally-managed network is an integral part of the SoS. It is also important managerially as it increases the difficulties of communication between those involved in making system management decisions and adds to the difficulties of maintaining system security.

Two further characteristics to Maier's list that are particularly relevant to systems of software systems:

1. **Data Intensive.** A software SoS typically relies on and manages a very large volume of data. In terms of size, this may be tens or even hundreds of times larger than the code of the constituent systems itself.
2. **Heterogeneity.** The different systems in a software SoS are unlikely to have been developed using the same programming languages and design methods. This is a consequence of the very rapid pace of evolution of software technologies. Companies frequently update their development methods and tools as new, improved versions become available. In a 20-year lifetime of a large SoS, technologies may change four or five times.



Learning Outcomes

At the end of this module, students should be able to:

1. Examine systems complexity and several different types of complexity are relevant to sociotechnical system;
2. Identify sociotechnical system of system classifications;
3. Differentiate systems engineering and architecture.

Lesson 1. System Complexity

All systems are composed of parts (elements) with relationships between these elements of the system. For example, the parts of a program may be objects, and the parts of each object may be constants, variables, and methods. Examples of relationships include “calls” (method A calls method B), “inherits-from” (object X inherits the methods and attributes of object Y), and “part of” (method A is part of object X).

The complexity of any system depends on the number and types of relationships between system elements. Figure 6.1 shows examples of two systems (Sommerville, n.d.):

1. System (a) is a relatively simple system with only a small number of relationships between its elements and by contrast;
2. System (b), with the same number of elements, is a more complex system because it has many more element–element relationships.

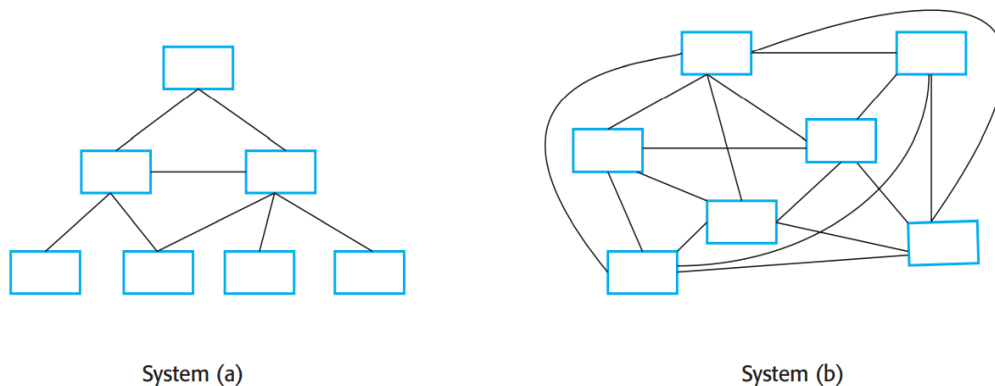


Figure 6.1 Simple and Complex Systems

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

The type of relationship also influences the overall complexity of a system. Static relationships are relationships that are planned and analyzable from static depictions of the system. Therefore, the “uses” relationship in a software system is a static relationship. From either the software source code or a UML model of a system, you can work out how anyone software component uses other components (Sommerville, n.d.).

Dynamic relationships are relationships that exist in an executing system. The “calls” relationship is a dynamic relationship because, in any system with if-statements, you cannot tell whether or not one method will call another method. It depends on the runtime inputs to

the system. Dynamic relationships are more complex to analyze as you need to know the system inputs and data used as well as the source code of the system.

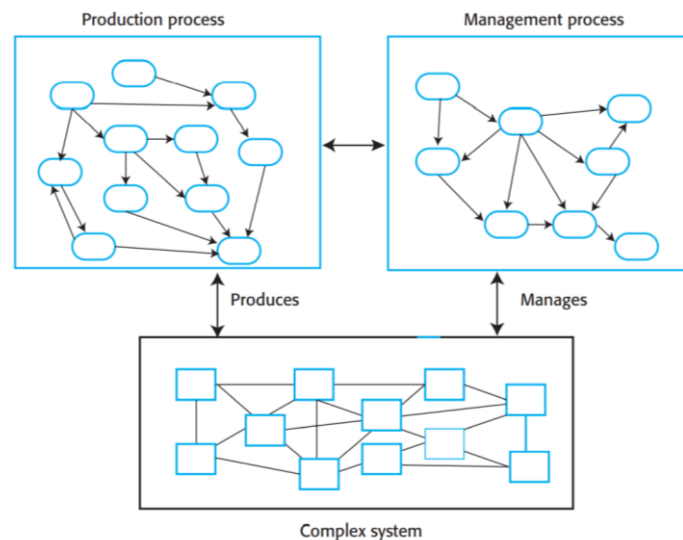


Figure 6.2 Production and Management Processes

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

As well as system complexity, we also have to consider the complexity of the processes used to develop and maintain the system once it has gone into use. Figure 6.2 illustrates these processes and their relationship with the developed system.

As systems grow in size, they need more complex production and management processes. Complex processes are themselves complex systems. They are difficult to understand and may have undesirable emergent properties. They are more time consuming than simpler processes, and they require more documentation and coordination between the people and the organizations involved in the system development. The complexity of the production process is one of the main reasons why projects go wrong, with software delivered late and overbudget. Therefore, large systems are always at risk of cost and time overruns (Sommerville, n.d.).

Complexity is important for software engineering because it is the main influence on the understandability and the changeability of a system. The more complex a system, the more difficult it is to understand and analyze. Given that complexity is a function of the number of relationships between elements of a system, it is inevitable that large systems are more complex than small systems. As complexity increases, there are more and more relationships between elements of the system and an increased likelihood that changing one part of a system will have undesirable effects elsewhere (Sommerville, n.d.).

Several different types of complexity are relevant to sociotechnical systems (Sommerville, n.d.):

1. The technical complexity of the system is derived from the relationships between the different components of the system itself.
2. The managerial complexity of the system is derived from the complexity of the relationships between the system and its managers (i.e., what can managers change in the system) and the relationships between the managers of different parts of the system.
3. The governance complexity of a system depends on the relationships between the laws, regulations, and policies that affect the system and the relationships between the decision-making processes in the organizations responsible for the system. As different parts of the system may be in different organizations and in different countries, different laws, rules, and policies may apply to each system within the SoS.

Governance and managerial complexity are related, but they are not the same thing. Managerial complexity is an operational issue what can and can't actually be done with the system. Governance complexity is associated with the higher level of decision-making processes in organizations that affect the system. These decisionmaking processes are constrained by national and international laws and regulations.

Lesson 2. Systems of Systems Classification

The distinguishing feature of a system of systems was that two or more of its elements were independently managed. Different people with different priorities have the authority to take day-to-day operational decisions about changes to the system. As their work is not necessarily aligned, conflicts can arise that require a significant amount of time and effort to resolve (Sommerville, n.d.).

Systems of systems, therefore, always have some degree of managerial complexity. However, this broad definition of SoS covers a very wide range of system types. It includes systems that are owned by a single organization but are managed by different parts of that organization. It also includes systems whose constituent systems are owned and managed by different organizations that may, at times, compete with each other.

Maier (1998) devised a classification scheme for SoS based on their governance and management complexity:

1. **Directed Systems.** Directed SoS are owned by a single organization and are developed by integrating systems that are also owned by that organization. The system elements may be independently managed by parts of the organization. However, there is an ultimate governing body within the organization that can set priorities for system management.
2. **Collaborative Systems.** Collaborative SoS are systems with no central authority to set management priorities and resolve disputes. Typically, elements of the system are owned and governed by different organizations. However, all of the organizations involved recognize the mutual benefits of joint governance of the system.
3. **Virtual Systems.** Virtual systems have no central governance, and the participants may not agree on the overall purpose of the system. Participant systems may enter or leave the SoS. Interoperability is not guaranteed but depends on published interfaces that may change. These systems have a very high degree of both managerial and governance complexity.

Lesson 3. Reductionism and Complex Systems

In software engineering, we have seen the incredibly rapid development of the discipline to help manage the increasing size and complexity of software systems. This effort has been very successful indeed. We can now build systems that are orders of magnitude larger and more complex than those of the 1960s and 1970s (Sommerville, n.d.).

As with other engineering disciplines, the approach that has been the basis of complexity management in software engineering is called reductionism. Reductionism is a philosophical position based on the assumptions that any system is made up of parts or subsystems. It assumes that the behavior and properties of the system as a whole can be understood and predicted by understanding the individual parts and the relationships between these parts. Therefore, to design a system, the parts making up that system are identified, constructed separately, and then assembled into the complete system.

Systems can be thought of as hierarchies, with the important relationships between parent and child nodes in the hierarchy. Reductionism has been and continues to be the

fundamental underpinning approach to all kinds of engineering. We can identify common abstractions across the same types of system and design and build these separately. They can then be integrated to create the required system.

For example, the abstractions in an automobile might be a body shell, a drive train, an engine, a fuel system, and so on. There are a relatively small number of relationships between these abstractions, so it is possible to specify interfaces and design and build each part of the system separately.

The same reductionist approach has been the basis of software engineering for almost 50 years. Top-down design, where you start with a very high-level model of a system and break this down to its components is a reductionist approach. This is the basis of all software design methods, such as object-oriented design. Programming languages include abstractions, such as procedures and objects that directly reflect reductionist system decomposition (Sommerville, n.d.).

Agile methods, although they may appear quite different from top-down systems design, are also reductionist. They rely on being able to decompose a system into parts, implement these parts separately, and then integrate these to create the system. The only real difference between agile methods and top-down design is that the system is decomposed into components incrementally rather than all at once.

Reductionist methods are most successful when there are relatively few relationships or interactions between the parts of a system and it is possible to model these relationships in a scientific way. This is generally true for mechanical and electrical systems where there are physical linkages between the system components. It is less true for electronic systems and certainly not the case for software systems, where there may be many more static and dynamic relationships between system components (Sommerville, n.d.).

The distinctions between software and hardware components was recognized in the 1970s. Design methods emphasized the importance of limiting and controlling the relationships between the parts of a system. These methods suggested that components should be tightly integrated with loose coupling between these components. Tight integration meant that most of the relationships were internal to a component, and loose coupling meant that there were relatively few component–component relationships. The need for tight integration (data and operations) and loose coupling was the driver for the development of object-oriented software engineering (Sommerville, n.d.).

The reasons for these potential difficulties are that the fundamental assumptions inherent to reductionism are inapplicable for large and complex systems (Sommerville et al. 2012). These assumptions are applied in three areas:

1. **System Ownership and Control.** Reductionism assumes that there is a controlling authority for a system that can resolve disputes and make high-level technical decisions that will apply across the system. As we have seen, because there are multiple bodies involved in their governance, this is simply not true for systems of systems.
2. **Rational Decision-Making.** Reductionism assumes that interactions between components can be objectively assessed by, for example, mathematical modeling. These assessments are the driver for system decision making. Therefore, if one particular design of a vehicle, say, offers the best fuel economy without a reduction in power, then a reductionist approach assumes that this will be the design chosen.
3. **Defined System Boundaries.** Reductionism assumes that the boundaries of a system can be agreed to and defined. This is often straightforward: There may be a physical shell defining the system as in a car, a bridge has to cross a given stretch of water, and so on.

Complex systems are often developed to address wicked problems (Rittel and Webber 1973). For such problems, deciding on what is part of the system and what is outside it is usually a subjective judgment, with frequent disagreements between the stakeholders involved.

These reductionist assumptions break down for all complex systems, but when these systems are software-intensive, the difficulties are compounded:

1. Relationships in software systems are not governed by physical laws. We cannot produce mathematical models of software systems that will predict their behavior and attributes. We therefore have no scientific basis for decision making. Political factors are usually the driver of decision making for large and complex software systems.
2. Software has no physical limitations; hence there are no limits on where the boundaries of a system should be drawn. Different stakeholders will argue for the boundaries to be placed in such a way that is best for them. Furthermore, it is much easier to change software requirements than hardware requirements. The boundaries and the scope of a system are likely to change during its development.

3. Linking software systems from different owners is relatively easy; hence we are more likely to try and create a SoS where there is no single governing body. The management and evolution of the different systems involved cannot be completely controlled.

Reductionist approaches such as object-oriented development have been very successful in improving our ability to engineer many types of software system. They will continue to be useful and effective in developing small and medium-sized systems whose complexity can be controlled and which may be parts of a software SoS (Sommerville, n.d.).

Lesson 4. Systems of Systems Engineering

Systems of systems engineering is the process of integrating existing systems to create new functionality and capabilities. Systems of systems are not designed in a top-down way (Sommerville, n.d.).

The problems of software SoS engineering has much in common with the problems of integrating large-scale application systems (Boehm and Abts 1999) these were:

1. Lack of control over system functionality and performance.
2. Differing and incompatible assumptions made by the developers of the different systems.
3. Different evolution strategies and timetables for the different systems.
4. Lack of support from system owners when problems arise.

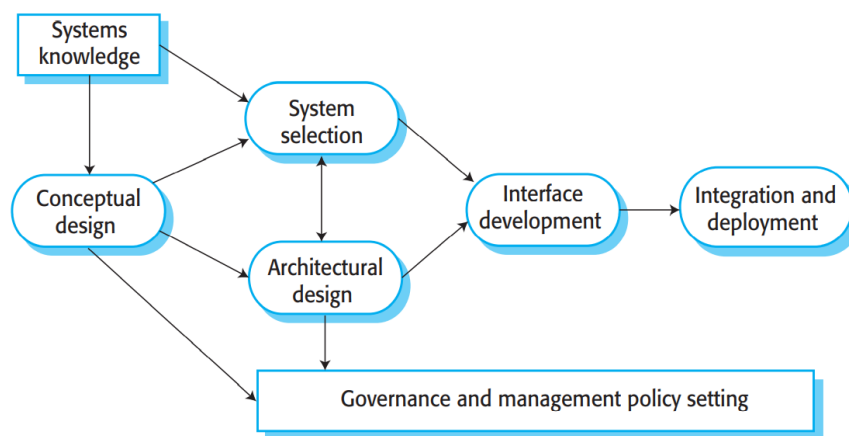


Figure 6.3 Systems of Systems Engineering Processes

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Much of the effort in building systems of software systems comes from addressing these problems. It involves deciding on the system architecture, developing software interfaces that reconcile differences between the participating systems, and making the system resilient to unforeseen changes that may occur (Sommerville, n.d.).

Software systems of systems are large and complex entities, and the processes used for their development vary widely depending on the type of systems involved, the application domain, and the needs of the organizations involved in developing the SoS.

Five General Activities Involved in SoS Development Processes (Sommerville, n.d.):

1. **Conceptual Design.** During this activity, creating a high-level vision for a system, defining essential requirements, and identifying constraints on the overall system. In SoS engineering, an important input to the conceptual design process is knowledge of the existing systems that may participate in the SoS.
2. **System Selection.** During this activity, a set of systems for inclusion in the SoS is chosen. This process is comparable to the process of choosing application systems for reuse. Existing systems are needed to be assessed and evaluated to choose the capabilities that is needed. When selecting application systems, the selection criteria are largely commercial; that is, which systems offer the most suitable functionality.
3. **Architectural Design.** In parallel with system selection, an overall architecture for the SoS has to be developed.
4. **Interface Development.** The different systems involved in a SoS usually have incompatible interfaces. Therefore, a major part of the software engineering effort in developing a SoS is to develop interfaces so that constituent systems can interoperate. This may also involve the development of a unified user interface so that SoS operators do not have to deal with multiple user interfaces as they use the different systems in the SoS.
5. **Integration and Deployment.** This stage involves making the different systems involved in the SoS work together and interoperate through the developed interfaces. System deployment means putting the system into place in the organizations concerned and making it operational.

Interface Development

The constituent systems in a SoS are usually developed independently for some specific purpose. Their user interface is tailored to that original purpose. These systems may or may not have application programming interfaces (APIs) that allow other systems to interface directly to them. Therefore, when these systems are integrated into a SoS, software interfaces have to be developed, which allows the constituent systems in the SoS to interoperate. In general, the aim in SoS development is for systems to be able to communicate directly with each other without user intervention (Sommerville, n.d.).

If these systems already offer a service-based interface, then this communication can be implemented using this approach. Interface development involves describing how to use the interfaces to access the functionality of each system. The systems involved can communicate directly with each other. System coalitions, where all of the systems involved are peers, are likely to use this type of direct interaction as it does not require prearranged agreements on system communication protocols (Sommerville, n.d.).

To develop service-based interfaces, you have to examine the functionality of existing systems and define a set of services to reflect that functionality. The interface then provides these services. User interfaces for each system in a SoS are likely to be different. The principal system must have some overall user interfaces that handle user authentication and provide access to the features of the underlying system. However, it is usually expensive and time consuming to implement a unified user interface to replace the individual interfaces of the underlying systems. A unified user interface (UI) makes it easier for new users to learn to use the SoS and reduces the likelihood of user error. However, whether or not unified UI development is cost-effective depends on a number of factors (Sommerville, n.d.):

1. **The interaction assumptions of the systems in the SoS.** Some systems may have a process-driven model of interaction where the system controls the interface and prompts the user for inputs. Others may give control to the user, so that the user chooses the sequence of interactions with the system. It is practically impossible to unify different interaction models.
2. **The mode of use of the SoS.** In many cases, SoS are used in such a way that most of the interactions of users at a site are with one of the constituent systems. They use other systems only when additional information is required. For example, air traffic controllers may normally use a radar system for flight information and only access a flight plan database when additional information is required. A unified interface is a bad idea in these

situations because it would slow down interaction with the most commonly used system. However, if the operators interact with all of the constituent systems, then a unified UI may be the best way forward.

3. **The “openness” of the SoS.** If the SoS is open, so that new systems may be added to it when it is in use, then unified UI development is impractical. It is impossible to anticipate what the UI of new systems will be. Openness also applies to the organizations using the SoS. If new organizations can become involved, then they may have existing equipment and their own preferences for user interaction. They may therefore prefer not to have a unified UI.

In practice, the limiting factor in UI unification is likely to be the budget and time available for UI development. UI development is one of the most expensive systems engineering activities. In many cases, there is simply not enough project budget available to pay for the creation of a unified SoS user interface (Sommerville, n.d.).

Integration and Deployment

System integration and deployment are usually separate activities. A system is integrated from its components by an integration and testing team, validated, and then released for deployment. The components are managed so that changes are controlled and the integration team can be confident that the required version is included in the system (Sommerville, n.d.).

However, for SoS, such an approach may not be possible. Some of the component systems may already be deployed and in use, and the integration team cannot control changes to these systems.

For SoS, therefore, it makes sense to consider integration and deployment to be part of the same process. This approach reflects one of the design guidelines that I discuss in the following section, which is that an incomplete system of systems should be usable and provide useful functionality. The integration process should begin with systems that are already deployed, with new systems added to the SoS to provide coherent additions to the functionality of the overall system (Sommerville, n.d.).

It often makes sense to plan the deployment of the SoS to reflect this, so that SoS deployment takes place in a number of stages. For example, Figure 6.4 illustrates a three-stage deployment process for the iLearn digital learning environment (Sommerville, n.d.):

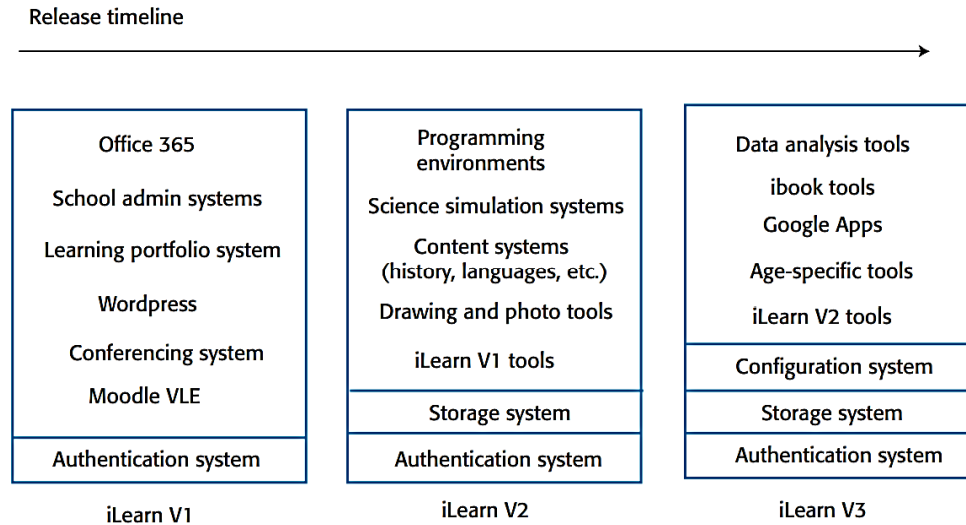


Figure 6.4 Release Sequence for the iLearn SoS

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Three-Stage Deployment Process (Sommerville, n.d.)

1. The initial deployment provides authentication, basic learning functionality, and integration with school administration systems.
2. Stage 2 of the deployment adds an integrated storage system and a set of more specialized tools to support subject-specific learning. These tools might include archives for history, simulation systems for science, and programming environments for computing.
3. Stage 3 adds features for user configuration and the ability of users to add new systems to the iLearn environment. This stage allows different versions of the system to be created for different age groups, further specialized tools, and alternatives to the standard tools to be included.

As in any large systems engineering project, the most time-consuming and expensive part of system integration is system testing. Testing systems of systems is difficult and expensive for three reasons (Sommerville, n.d.):

1. There may not be a detailed requirements specification that can be used as a basis for system testing. It may not be cost-effective to develop a SoS requirements document because the details of the system functionality are defined by the systems that are included.

2. The constituent systems may change in the course of the testing process, so tests may not be repeatable.
3. If problems are discovered, it may not be possible to fix the problems by requiring one or more of the constituent systems to be changed. Rather, some intermediate software may have to be introduced to solve the problem.

To help address some of these problems, SoS testing should take on board some of the testing techniques developed in agile methods (Sommerville, n.d.):

1. Agile methods do not rely on having a complete system specification for system acceptance testing. Rather, stakeholders are closely engaged with the testing process and have the authority to decide when the overall system is acceptable. For SoS, a range of stakeholders should be involved in the testing process if possible, and they can comment on whether or not the system is ready for deployment.
2. Agile methods make extensive use of automated testing. This makes it much easier to rerun tests to discover if unexpected system changes have caused problems for the SoS as a whole. Depending on the type of system, you may have to plan the installation of equipment and user training as part of the deployment process.

If the system is being installed in a new environment, equipment installation is straightforward. However, if it is intended to replace an existing system, there may be problems in installing new equipment if it is not compatible with the equipment that is in use. There may not be the physical space for the new equipment to be installed alongside the working system. There may be insufficient electrical power, or users may not have time to be involved because they are busy using the current system. These nontechnical issues can delay the deployment process and slow down the adoption and use of the SoS (Sommerville, n.d.).

Lesson 5. Systems of Systems Architecture

Perhaps the most crucial activity of the systems of systems engineering process is architectural design. Architectural design involves selecting the systems to be included in the

SoS, assessing how these systems will interoperate, and designing mechanisms that facilitate interaction.

Key decisions on data management, redundancy, and communications are made. In essence, the SoS architect is responsible for realizing the vision set out in the conceptual design of the system. For organizational and federated systems, in particular, decisions made at this stage are crucial to the performance, resilience, and maintainability of the system of systems.

Maier (1998) discusses four general principles for the architecting of complex systems of systems:

1. **Design systems so that they can deliver value if they are incomplete.** Where a system is composed of several other systems, it should not just be useful if all of its components are working properly. Rather, there should be several “stable intermediate forms” so that a partial system works and can-do useful things.
2. **Be realistic about what can be controlled.** The best performance from a SoS may be achieved when an individual or group exerts control over the overall system and its constituents. If there is no control, then delivering value from the SoS is difficult. However, attempts to overcontrol the SoS are likely to lead to resistance from the individual system owners and consequent delays in system deployment and evolution.
3. **Focus on the system interfaces.** To build a successful system of systems, you have to design interfaces so that the system elements can interoperate. It is important that these interfaces are not too restrictive so that the system elements can evolve and continue to be useful participants in the SoS.
4. **Provide collaboration incentives.** When the system elements are independently owned and managed, it is important each system owner have incentives to continue to participate in the system. These may be financial incentives (pay per use or reduced operational costs), access incentives (you share your data and I’ll share mine), or community incentives (participate in a SoS and you get a say in the community).

Sillitto (2010) has added to these principles and suggests additional important design guidelines. These include the following:

1. **Design a SoS as node and web architecture.** Nodes are sociotechnical systems that include data, software, hardware, infrastructure (technical components), and organizational policies, people, processes, and training (sociotechnical). The web is not just the communications infrastructure between nodes, but it also provides a mechanism for informal and formal social communications between the people managing and running the systems at each node.
2. **Specify behavior as services exchanged between nodes.** The development of service-oriented architectures now provides a standard mechanism for system operability. If a system does not already provide a service interface, then this interface should be implemented as part of the SoS development process.
3. **Understand and manage system vulnerabilities.** In any SoS, there will be unexpected failures and undesirable behavior. It is critically important to try to understand vulnerabilities and design the system to be resilient to such failures.

The key message that emerges from both Maier's and Sillitto's work is that SoS architects have to take a broad perspective. They need to look at the system as a whole, taking into account both technical and sociotechnical considerations. Sometimes the best solution to a problem is not more software but changes to the rules and policies that govern the operation of the system

MODULE 7

REAL-TIME SOFTWARE ENGINEERING



Introduction

Computers are used to control a wide range of systems from simple domestic machines, through games controllers, to entire manufacturing plants. These computers interact directly with hardware devices. Their software must react to events generated by the hardware and often issue control signals in response to these events. These signals result in an action, such as the initiation of a phone call, the movement of a character on the screen, the opening of a valve, or the display of the system status (Sommerville, n.d.).

The software in these systems is embedded in system hardware, often in read-only memory. It responds, in real time, to events from the system's environment. By real time, it means that the software system has a deadline for responding to external events. If this deadline is missed, then the overall hardware–software system will not operate correctly.

Embedded software is very important economically because almost every electrical device now includes software. There are therefore many more embedded software systems than other types of software systems.

Responsiveness in real time is the critical difference between embedded systems and other software systems, such as information systems, web-based systems, or personal software systems, whose main purpose is data processing. For non–real-time systems, the correctness of a system can be defined by specifying how system inputs map to corresponding outputs that should be produced by the system (Sommerville, n.d.).

In response to an input, a corresponding output should be generated by the system and, often, some data should be stored. For example, if you choose a create command in a patient information system, then the correct system response is to create a new patient record in a database and to confirm that this has been done.

Within reasonable limits, it does not matter how long this takes. However, in a real-time system, the correctness depends both on the response to an input and the time taken to generate that response. If the system takes too long to respond, then the required response may be ineffective.

For example, if embedded software controlling a car's braking system is too slow, then an accident may occur because it is impossible to stop the car in time. Therefore, time is fundamental in the definition of a real-time software system (Sommerville, n.d.):

A real-time software system is a system whose correct operation depends on both the results produced by the system and the time at which these results are produced. A "soft real-time system" is a system whose operation is degraded if results are not produced according to the specified timing requirements. If results are not produced according to the timing specification in a "hard realtime system," this is considered to be a system failure.

Timely response is an important factor in all embedded systems, but not all embedded systems require a very fast response. For example, the insulin pump software that I have used as an example in several chapters of this book is an embedded system. However, while the system needs to check the glucose level at periodic intervals, it does not need to respond very quickly to external events.

The wilderness weather station software is also an embedded system, but, again, it does not require a fast response to external events. As well as the need for real-time response, there are other important differences between embedded systems and other types of software system (Sommerville, n.d.):

1. **Embedded systems generally run continuously and do not terminate.** They start when the hardware is switched on, and execute until the hardware is switched off. Techniques for reliable software engineering, as discussed in the previous lessons, may therefore have to be used to ensure continuous operation. The real-time system may include update mechanisms that support dynamic reconfiguration so that the system can be updated while it is in service.
2. **Interactions with the system's environment are unpredictable.** In interactive systems, the pace of the interaction is controlled by the system. By limiting user options, the events and commands to be processed are known in advance. By contrast, real-time embedded systems must be able to respond to expected and unexpected events at any time. This leads to a design for real-time systems based on concurrency, with several processes executing in parallel.

3. **Physical limitations may affect the design of a system.** Examples of limitations include restrictions on the power available to the system and the physical space taken up by the hardware. These limitations may generate requirements for the embedded software, such as the need to conserve power and so prolong battery life. Size and weight limitations may mean that the software has to take over some hardware functions because of the need to limit the number of chips used in the system.
4. **Direct hardware interaction may be necessary.** In interactive systems and information systems, a layer of software (the device drivers) hides the hardware from the operating system. This is possible because you can only connect a few types of device to these systems, such as keyboards, mice, and displays. By contrast, embedded systems may have to interact with a wide range of hardware devices that do not have separate device drivers.
5. **Issues of safety and reliability may dominate the system design.** Many embedded systems control devices whose failure may have high human or economic costs. Therefore, dependability is critical, and the system design has to ensure safety-critical behavior at all times. This often leads to a conservative approach to design where tried and tested techniques are used instead of newer techniques that may introduce new failure modes.

Real-time embedded systems can be thought of as reactive systems; that is, they must react to events in their environment (Berry 1989; Lee 2002). Response times are often governed by the laws of physics rather than chosen for human convenience. This is in contrast to other types of software where the system controls the speed of the interaction. For example, the word processor that I am using to write this book can check spelling and grammar, and there are no practical limits on the time taken to do so.

Learning Outcomes

At the end of this lesson, the student should be able to:

1. understand the Embedded systems design, Architectural patterns for real-time software, Timing analysis and Real-time operating systems

Lesson 1. Embedded Systems Design

During the design process for embedded software, software designers have to consider in detail the design and performance of the system hardware. Part of the system design process may involve deciding which system capabilities are to be implemented in software and which in hardware. For many real-time systems that are embedded in consumer products, such as the systems in cell phones, the costs and power consumption of the hardware are critical (Sommerville, n.d.).

Specific processors designed to support embedded systems may be used. For some systems, special-purpose hardware may have to be designed and built. A top-down software design process, in which the design starts with an abstract model that is decomposed and developed in a series of stages, is impractical for most real-time systems. Low-level decisions on hardware, support software, and system timing must be considered early in the process.

These limit the flexibility of system designers. Additional software functionality, such as battery and power management, may have to be included in the system. Given that embedded systems are reactive systems that react to events in their environment, the most general approach to embedded, real-time software design is based on a stimulus-response model.

A stimulus is an event occurring in the software system's environment that causes the system to react in some way; a response is a signal or message that the software sends to its environment. You can define the behavior of a real-time system by listing the stimuli received by the system, the associated responses, and the time at which the response must be produced.

For example, Figure 3.1 below shows possible stimuli and system responses for a burglar alarm system. Stimuli fall into two classes (Sommerville, n.d.):

1. **Periodic Stimuli.** These occur at predictable time intervals. For example, the system may examine a sensor every 50 milliseconds and take action (respond) depending on that sensor value (the stimulus).
2. **Aperiodic Stimuli.** These occur irregularly and unpredictably and are usually signaled, using the computer's interrupt mechanism. An example of such a stimulus would be an interrupt indicating that an I/O transfer was complete and that data was available in a buffer.

Stimuli come from sensors in the system's environment, and responses are sent to actuators, as shown in Figure 3.2. These actuators control equipment, such as a pump, which then makes changes to the system's environment. The actuators themselves may also generate stimuli. The stimuli from actuators often indicate that some problem with the actuator has occurred, which must be handled by the system (Sommerville, n.d.).

A general design guideline for real-time systems is to have separate control processes for each type of sensor and actuator (Figure 7.1). For each type of sensor, there may be a sensor management process that handles data collection from these sensors. Data-processing processes compute the required responses for the stimuli received by the system.

Stimulus	Response
Clear alarms	Switch off all active alarms; switch off all lights that have been switched on.
Console panic button positive	Initiate alarm; turn on lights around console; call police.
Power supply failure	Call service technician.
Sensor failure	Call service technician.
Single sensor positive	Initiate alarm; turn on lights around site of positive sensor.
Two or more sensors positive	Initiate alarm; turn on lights around sites of positive sensors; call police with location of suspected break-in.
Voltage drop of between 10% and 20%	Switch to battery backup; run power supply test.
Voltage drop of more than 20%	Switch to battery backup; initiate alarm; call police, run power supply test.

Figure 7.1 Stimuli and Responses to a Burglar Alarm System

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Actuator control processes are associated with each actuator and manage the operation of that actuator. This model allows data to be collected quickly from the sensor (before it is overwritten by the next input) and enables processing and the associated actuator response to be carried out later.

A real-time system has to respond to stimuli that occur at different times. You therefore have to organize the system architecture so that, as soon as a stimulus is received, control is transferred to the correct handler. This is impractical in sequential programs. Consequently, real-time software systems are normally designed as a set of concurrent, cooperating

processes. To support the management of these processes, the execution platform on which the real-time system executes may include a real-time operating system.

The functions provided by this operating system are accessed through the runtime support system for the realtime programming language that is used. There is no standard embedded system design process. Rather, different processes are used that depend on the type of system, available hardware, and the organization that is developing the system. The following activities may be included in a real-time software design process (Sommerville, n.d.):

1. **Platform Selection.** In this activity, you choose an execution platform for the system, that is, the hardware and the real-time operating system to be used. Factors that influence these choices include the timing constraints on the system, limitations on power available, the experience of the development team, and the price target for the delivered system.
2. **Stimuli/Response Identification.** This involves identifying the stimuli that the system must process and the associated response or responses for each stimulus
3. **Timing Analysis.** For each stimulus and associated response, you identify the timing constraints that apply to both stimulus and response processing. These constraints are used to establish the deadlines for the processes in the system.
4. **Process Design.** Process design involves aggregating the stimulus and response processing into a number of concurrent processes. A good starting point for designing the process architecture is the architectural patterns that I describe in Section 20.2. You then optimize the process architecture to reflect the specific requirements that you have to implement.
5. **Algorithm Design.** For each stimulus and response, you design algorithms to carry out the required computations. Algorithm designs may have to be developed relatively early in the design process to indicate the amount of processing required and the time needed to complete that processing. This is especially important for computationally intensive tasks, such as signal processing.
6. **Data Design.** You specify the information that is exchanged by processes and the events that coordinate information exchange, and design data structures to manage this information exchange. Several concurrent processes may share these data structures.
7. **Process Scheduling.** You design a scheduling system that will ensure that processes are started in time to meet their deadlines.

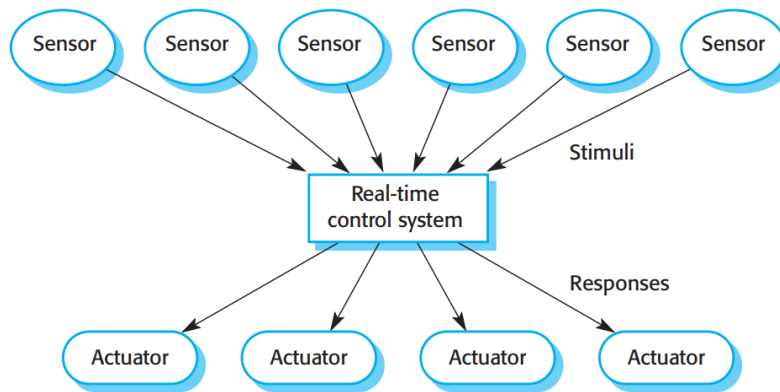


Figure 7.2 A General Model of an Embedded Real-time Systems

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

The specific activities and the activity sequence in a real-time system design process depend on the type of system being developed, its novelty, and its environment.

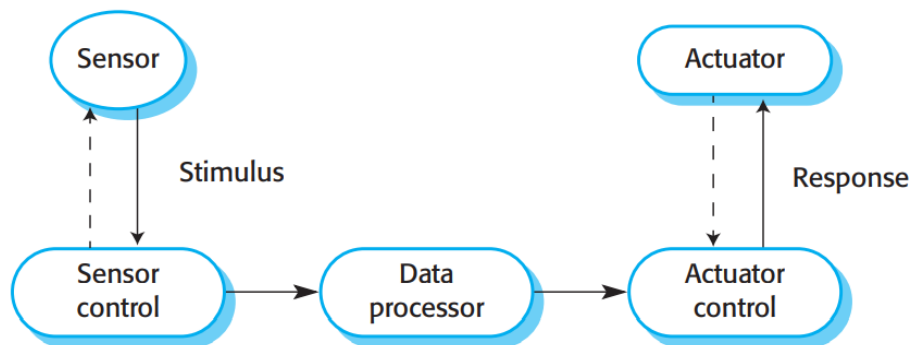


Figure 7.3 Sensor and Actuator Processes

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Real-Time System Modeling

The events that a real-time system must react to often cause the system to move from one state to another. For this reason, state models are used to describe real-time systems. A state model of a system assumes that, at any time, the system is in one of a number of possible states.

When a stimulus is received, this may cause a transition to a different state. For example, a system controlling a valve may move from a state “Valve open” to a state “Valve closed” when an operator command (the stimulus) is received. State models are an integral part of real-time system design methods.

The UML supports the development of state models based on Statecharts (Harel 1987, 1988). Statecharts are formal state machine models that support hierarchical states, so that groups of states can be considered as a single entity.

Douglass (1999) discusses the use of the UML in real-time systems development. Figure 7.4 is another example of a state model that shows the operation of a fuel delivery software system embedded in a petrol (gas) pump. The rounded rectangles represent system states, and the arrows represent stimuli that force a transition from one state to another.

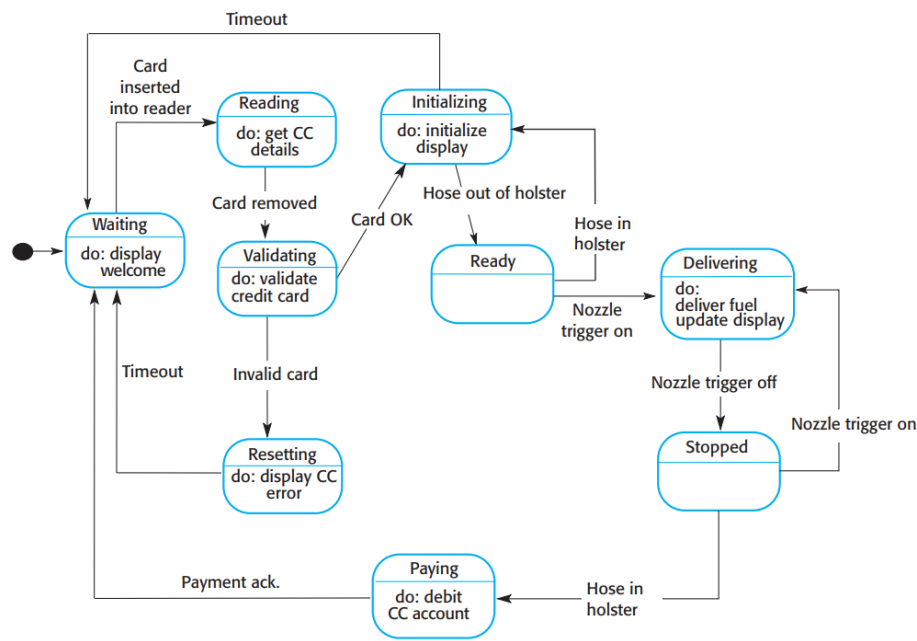


Figure 7.4 State Machine Model of a Petrol (Gas) Pump

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

The names chosen in the state machine diagram are descriptive. The associated information indicates actions taken by the system actuators or information that is displayed. Notice that this system never terminates but idles in a waiting state when the pump is not operating. The fuel delivery system is designed to allow unattended operation, with the following sequence of actions (Sommerville, n.d.):

1. The buyer inserts a credit card into a card reader built into the pump. This causes a transition to a Reading state where the card details are read and the buyer is then asked to remove the card.
2. Removal of the card triggers a transition to a Validating state where the card is validated.

3. If the card is valid, the system initializes the pump and, when the fuel hose is removed from its holster, transitions to the Delivering state, where is ready to deliver fuel. Activating the trigger on the nozzle causes fuel to be pumped; this stops when the trigger is released (for simplicity, ignore the pressure switch that is designed to stop fuel spillage).
4. After the fuel delivery is complete and the buyer has replaced the hose in its holster, the system moves to a Paying state where the user's account is debited.
5. After payment, the pump software returns to the Waiting state. State models are used in model-driven engineering to define the operation of a system. They can be transformed automatically or semiautomatically to an executable program.

Real-time Programming



Real-time Java

The Java programming language has been modified to make it suitable for real-time systems development. These modifications include asynchronous communications, the addition of time, including absolute and relative time, a new thread model where threads cannot be interrupted by garbage collection, and a new memory management model that avoids the unpredictable delays that can result from garbage collection.

<http://software-engineering-book.com/web/real-time-java/>

Programming languages for real-time systems development have to include facilities to access system hardware, and it should be possible to predict the timing of particular operations in these languages. Hard real-time systems, running on limited hardware, are still sometimes programmed in assembly language so that tight deadlines can be met.

Systems programming languages, such as C, which allow efficient code to be generated, are widely used. The advantage of using a systems programming language like C is that it allows the development of efficient programs. However, these languages do not include constructs to support concurrency or the management of shared resources (Sommerville, 2019).

Concurrency and resource management are implemented through calls to primitives provided by the real-time operating system for mutual exclusion. Because the compiler cannot check these calls, programming errors are more likely.

Programs are also often more difficult to understand because the language does not include real-time features. As well as understanding the program, the reader also has to know how real-time support is provided using system calls. Because real-time systems must meet their timing constraints, you may not be able to use object-oriented development for hard real-time systems.

Object-oriented development involves hiding data representations and accessing attribute values through operations defined with the object. There is a significant performance overhead in object-oriented systems because extra code is required to mediate access to attributes and handle calls to operations. The consequent loss of performance may make it impossible to meet real-time deadlines.

A version of Java has been developed for embedded systems development (Burns and Wellings 2009; Bruno and Bollella 2009). This language includes a modified thread mechanism, which allows threads to be specified that will not be interrupted by the language garbage collection mechanism. Asynchronous event handling and timing specification has also been included. However, at the time of writing, this specification has mostly been used on platforms that have significant processor and memory capacity (e.g., a cell phone) rather than simpler embedded systems, with more limited resources. These systems are still usually implemented in C.

Lesson 2. Architectural Patterns for Real-Time Software

Architectural patterns are abstract, stylized descriptions of good design practice. They capture knowledge about the organization of system architectures, when these architectures should be used, and their advantages and disadvantages. You use an architectural pattern to understand an architecture and as starting point for creating your own, specific architectural design.

The difference between real-time and interactive software means that there are distinct architectural patterns for real-time embedded systems. Real-time systems' patterns are process-oriented rather than object- or component-oriented. The following are the three real-time architectural patterns that are commonly used:

1. **Observe and React.** This pattern is used when a set of sensors are routinely monitored and displayed. When the sensors show that some event has occurred (e.g., an incoming call on a cell phone), the system reacts by initiating a process to handle that event.
2. **Environmental Control.** This pattern is used when a system includes sensors, which provide information about the environment and actuators that can change the environment. In response to environmental changes detected by the sensor, control signals are sent to the system actuators.
3. **Process Pipeline.** This pattern is used when data has to be transformed from one representation to another before it can be processed. The transformation is implemented as a sequence of processing steps, which may be carried out concurrently. This allows for very fast data processing, because a separate core or processor can execute each transformation.

These patterns can of course be combined, and you will often see more than one of them in a single system. For example, when the Environmental Control pattern is used, it is very common for the actuators to be monitored using the Observe and React pattern.

Name	Observe and React
Description	The input values of a set of sensors of the same types are collected and analyzed. These values are displayed in some way. If the sensor values indicate that some exceptional condition has arisen, then actions are initiated to draw the operator's attention to that value and, if necessary, take actions in response to the exceptional value.
Stimuli	Values from sensors attached to the system.
Responses	Outputs to display, alarm triggers, signals to reacting systems.
Processes	Observer, Analysis, Display, Alarm, Reactor.
Used in	Monitoring systems, alarm systems.

Figure 7.5 The Observe and React Pattern

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

In the event of an actuator failure, the system may react by displaying a warning message, shutting down the actuator, switching in a backup system, and so forth. The patterns are architectural patterns that describe the overall structure of an embedded system.

Douglass (2002) describes lower-level, real-time design patterns that support more detailed design decision making. These patterns include design patterns for execution control, communications, resource allocation, and safety and reliability.

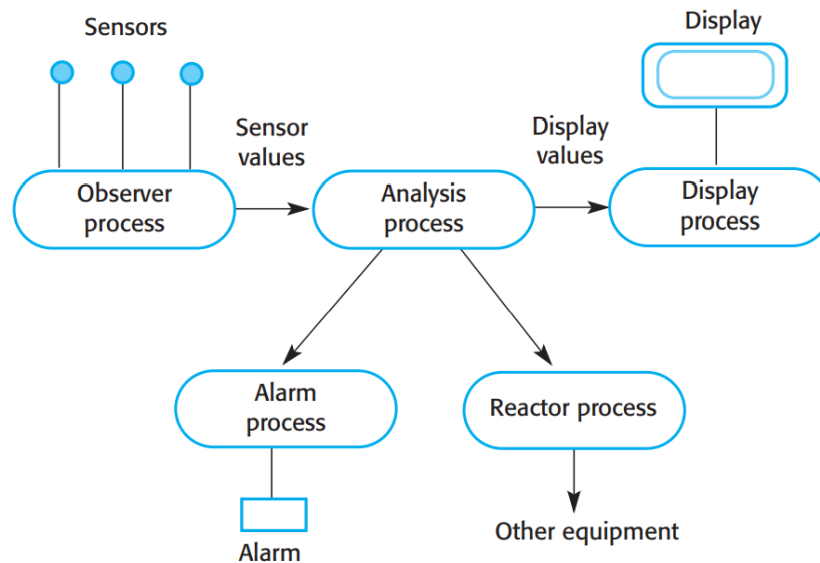


Figure 7.6 The Observe and React Process Structure

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

These architectural patterns should be the starting point for an embedded systems design; however, they are not design templates. If you use them as such, you will probably end up with an inefficient process architecture. You have to optimize the process structure to ensure that you do not have too many processes. You also should ensure that there is a clear correspondence between the processes and the sensors and actuators in the system (Sommerville, n.d.).

Observe and React

Monitoring systems are an important class of embedded real-time systems. A monitoring system examines its environment through a set of sensors and usually displays the state of the environment in some way. This could be on a built-in screen, on special-

purpose instrument displays, or on a remote display. If the system detects some exceptional event or sensor state, the monitoring system takes some action. This often involves raising an alarm to draw an operator's attention to the event. Sometimes the system may initiate some other preventative action, such as shutting down the system to preserve it from damage (Sommerville, n.d.).

The Observe and React pattern (Figures 7.5 and 7.6) is commonly used in monitoring systems. The values of sensors are observed, and the system initiates actions that depend on these sensor values.

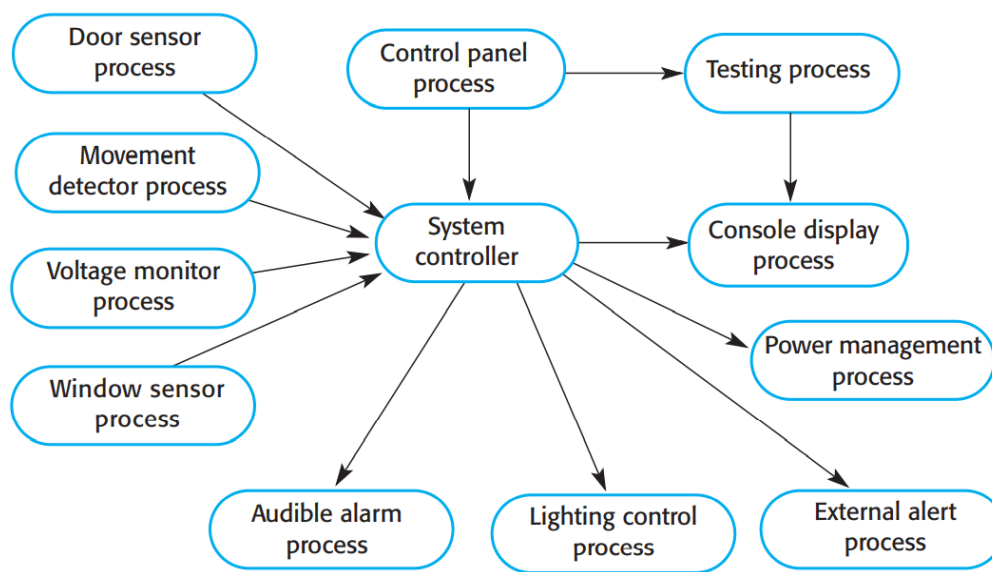


Figure 7.7 The Process Structure of a Burglar Alarm System

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Monitoring systems may be composed of several instantiations of the Observe and React pattern, one for each type of sensor in the system. Depending on the system requirements, you may then optimize the design by combining processes (e.g., you may use a single display process to display the information from all of the different types of sensor). As an example of the use of this pattern, consider the design of a burglar alarm system to be installed in an office building (Sommerville, n.d.):

A software system is to be implemented as part of a burglar alarm system for commercial buildings. This uses several different types of

sensors. These sensors include movement detectors in individual rooms, door sensors that detect corridor doors opening, and window sensors on ground-floor windows that can detect when a window has been opened.

When a sensor detects the presence of an intruder, the system automatically calls the local police and, using a voice synthesizer, reports the location of the alarm. It switches on lights in the rooms around the active sensor and sets off an audible alarm. The sensor system is normally powered by mains power but is equipped with a battery backup. Power loss is detected using a separate power circuit monitor that monitors the mains voltage. If a voltage drop is detected, the system assumes that intruders have interrupted the power supply, so an alarm is raised.

A process architecture for the alarm system is shown in Figure 7.7. The arrows represent signals sent from one process to another. This system is a “soft” real-time system that does not have stringent timing requirements. The sensors only need to detect the presence of people rather than high-speed events, so they only need to be polled 2 or 3 times per second.

Environmental Control

The most widespread use of real-time embedded software is in control systems. In these systems, the software controls the operation of equipment, based on stimuli from the equipment’s environment. For example, an anti-skid braking system in a car monitors the car’s wheels and brake system (the system’s environment).

It looks for signs that the wheels are skidding when brake pressure is applied. If this is the case, the system adjusts the brake pressure to stop the wheels locking and reduce the likelihood of a skid. Control systems may make use of the Environmental Control pattern, which is a general control pattern that includes sensor and actuator processes (Sommerville, n.d.).

Process Pipeline

Many real-time systems are concerned with collecting analog data from the system’s environment. They then digitize that data for analysis and processing by the system. The

system may also convert digital data to analog data, which it then sends to its environment. For example, a software radio accepts incoming packets of digital data representing the radio transmission and transforms the data into a sound signal that people can listen to (Sommerville, n.d.).

The data processing involved in many of these systems has to be carried out very quickly. Otherwise, incoming data may be lost and outgoing signals may be broken up because essential information is missing. The Process Pipeline pattern makes this rapid processing possible by breaking down the required data processing into a sequence of separate transformations. Each of these transformations is implemented by an independent process. This architecture is efficient for systems that use multiple processors or multicore processors. Each process in the pipeline can be associated with a separate processor or core, so that the processing steps can be carried out in parallel (Sommerville, n.d.).

An example of a system that may use a process pipeline is a high-speed data acquisition system. Data acquisition systems collect data from sensors for subsequent processing and analysis. These systems are used in situations where the sensors are collecting large volumes of data from the system's environment and it isn't possible or necessary to process that data in real time. Rather, it is collected and stored for later analysis.

Data acquisition systems are often used in scientific experiments and process control systems where physical processes, such as chemical reactions, are very rapid. In these systems, the sensors may be generating data very quickly, and the data acquisition system has to ensure that a sensor reading is collected before the sensor value changes (Sommerville, n.d.).

Lesson 3. Timing Analysis

The correctness of a real-time system depends not just on the correctness of its outputs but also on the time at which these outputs were produced. Therefore, timing analysis is an important activity in the embedded, real-time software development process. In such an analysis, you calculate how often each process in the system must be executed to ensure that all inputs are processed and all system responses are produced in a timely way.

The results of the timing analysis are used to decide how frequently each process should execute and how these processes should be scheduled by the real-time operating system. Timing analysis for real-time systems is particularly difficult when the system has to

deal with a mixture of periodic and aperiodic stimuli and responses. Because aperiodic stimuli are unpredictable, you have to make assumptions about the probability of these stimuli occurring and therefore requiring service at any particular time. These assumptions may be incorrect, and system performance after delivery may not be adequate (Sommerville, n.d.).

Cooling's book (2003) discusses techniques for real-time system performance analysis that takes aperiodic events into account. As computers have become faster, it has become possible in many systems to design using only periodic stimuli. When processors were slow, aperiodic stimuli had to be used to ensure that critical events were processed before their deadline, as delays in processing usually involved some loss to the system.

For example, the failure of a power supply in an embedded system may mean that the system has to shut down attached equipment in a controlled way, within a very short time (say 50 milliseconds). This could be implemented as a "power fail" interrupt. However, it can also be implemented using a periodic process that runs frequently and checks the power. As long as the time between process invocations is short, there is still time to perform a controlled shutdown of the system before the lack of power causes damage.

When you are analyzing the timing requirements of embedded real-time systems and designing systems to meet these requirements, you have to consider three key factors (Sommerville, n.d.):

1. **Deadlines.** The times by which stimuli must be processed and some response produced by the system. If the system does not meet a deadline, then, if it is a hard-real-time system, this is a system failure; in a soft real-time system, it results in degraded system service.
2. **Frequency.** The number of times per second that a process must execute so that you are confident that it can always meet its deadlines.
3. **Execution Time.** The time required to process a stimulus and produce a response. Execution time is not always the same because of the conditional execution of code, delays waiting for other processes, and so on. Therefore, you may have to consider both the average execution time of a process and the worst-case execution time for that process.

The worst-case execution time is the maximum time that the process takes to execute. In a hard-real-time system, you may have to make assumptions based on the worst-case execution time to ensure that deadlines are not missed. In soft real-time systems, you can base your calculations on the average execution time (Sommerville, n.d.).

Stimulus/Response	Timing requirements
Audible alarm	The audible alarm should be switched on within half a second of an alarm being raised by a sensor.
Communications	The call to the police should be started within 2 seconds of an alarm being raised by a sensor.
Door alarm	Each door alarm should be polled twice per second.
Lights switch	The lights should be switched on within half a second of an alarm being raised by a sensor.
Movement detector	Each movement detector should be polled twice per second.
Power failure	The switch to backup power must be completed within a deadline of 50 ms.
Voice synthesizer	A synthesized message should be available within 2 seconds of an alarm being raised by a sensor.
Window alarm	Each window alarm should be polled twice per second.

Figure 7.8 Timing Requirements for the Burglar Alarm System

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

As shown in Figure 7.8, you should list the timing constraints for each class of sensor separately, even when (as in this case) they are the same. By considering them separately, you leave scope for future change and make it easier to compute the number of times the controlling process has to be executed each second.

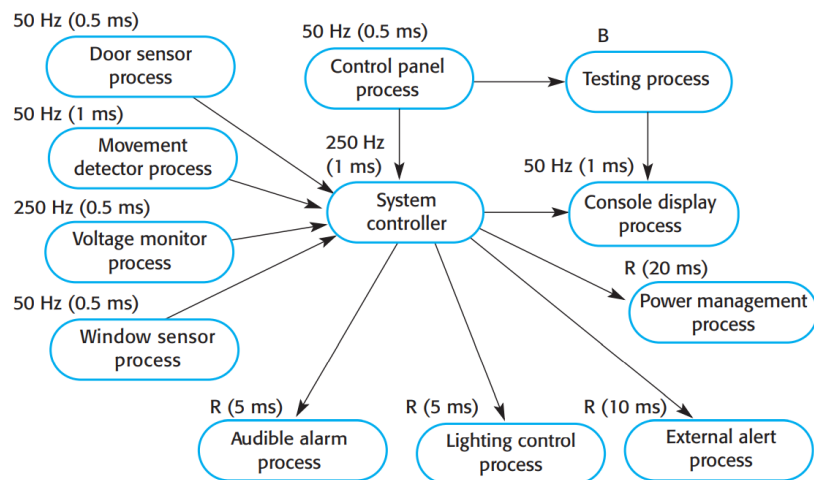


Figure 7.9 Alarm Process Timing

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Allocating the system functions to concurrent processes is the next design stage. Four types of sensors must be polled periodically, each with an associated process: the voltage sensor, door sensors, window sensors, and movement detectors. Normally, the processes associated with the sensor will execute very quickly as all they are doing is checking whether or not a sensor has changed its status (e.g., from off to on). It is reasonable to assume that the execution time to check and assess the state of one sensor is less than 1 millisecond (Sommerville, n.d.).

To ensure that you meet the deadlines defined by the timing requirements, you then have to decide how frequently the related processes have to run and how many sensors should be examined during each execution of the process. There are obvious trade-offs here between frequency and execution time (Sommerville, n.d.):

1. The deadline for detecting a change of state is 0.25 second, which means that each sensor has to be checked 4 times per second. If you examine one sensor during each process execution, then if there are N sensors of a particular type, you must schedule the process $4N$ times per second to ensure that all sensors are checked within the deadline.
2. If you examine four sensors, say, during each process execution, then the execution time is increased to about 4 ms, but you need only run the process N times/ second to meet the timing requirement.

The final step in the design process is to design a scheduling system that will ensure that a process will always be scheduled to meet its deadlines. You can only do this if you know the scheduling approaches that are supported by the real-time operating system (OS) used (Burns and Wellings, 2009).

The scheduler in the real-time OS allocates a process to a processor for a given amount of time. The time can be fixed, or it may vary depending on the priority of the process. In allocating process priorities, you have to consider the deadlines of each process so that processes with short deadlines receive processor time to meet these deadlines.

For example, the voltage monitor process in the burglar alarm needs to be scheduled so that voltage drops can be detected and a switch made to backup power before the system fails. This should therefore have a higher priority than the processes that check sensor values, as these have fairly relaxed deadlines compared to their expected execution time (Burns and Wellings, 2009).

Lesson 4. Real-Time Operating Systems

The execution platform for most application systems is an operating system that manages shared resources and provides features such as a file system and runtime process management. However, the extensive functionality in a conventional operating system takes up a great deal of space and slows down the operation of programs. Furthermore, the process management features in the system may not be designed to allow fine-grain control over the scheduling of processes (Sommerville, n.d.).

For these reasons, standard operating systems, such as Linux and Windows, are not normally used as the execution platform for real-time systems. Very simple embedded systems may be implemented as “bare metal” systems. The systems provide their own execution support and so include system startup and shutdown, process and resource management, and process scheduling.

More commonly, however, embedded applications are built on top of a real-time operating system (RTOS), which is an efficient operating system that offers the features needed by real-time systems. Examples of RTOS are Windows Embedded Compact, VxWorks, and RTLinux.

A real-time operating system manages processes and resource allocation for a real-time system. It starts and stops processes so that stimuli can be handled, and it allocates memory and processor resources. The components of an RTOS (Figure 21.18) depend on the size and complexity of the real-time system being developed. For all except the simplest systems, they usually include (Sommerville, n.d.):

1. A real-time clock, which provides the information required to schedule processes periodically.
2. If interrupts are supported, an interrupt handler, which manages aperiodic request for service
3. A scheduler, which is responsible for examining the processes that can be executed and for choosing one of these processes for execution.
4. A resource manager, which allocates appropriate memory and processor resources to processes that have been scheduled for execution.
5. A dispatcher, which is responsible for starting the execution of processes.

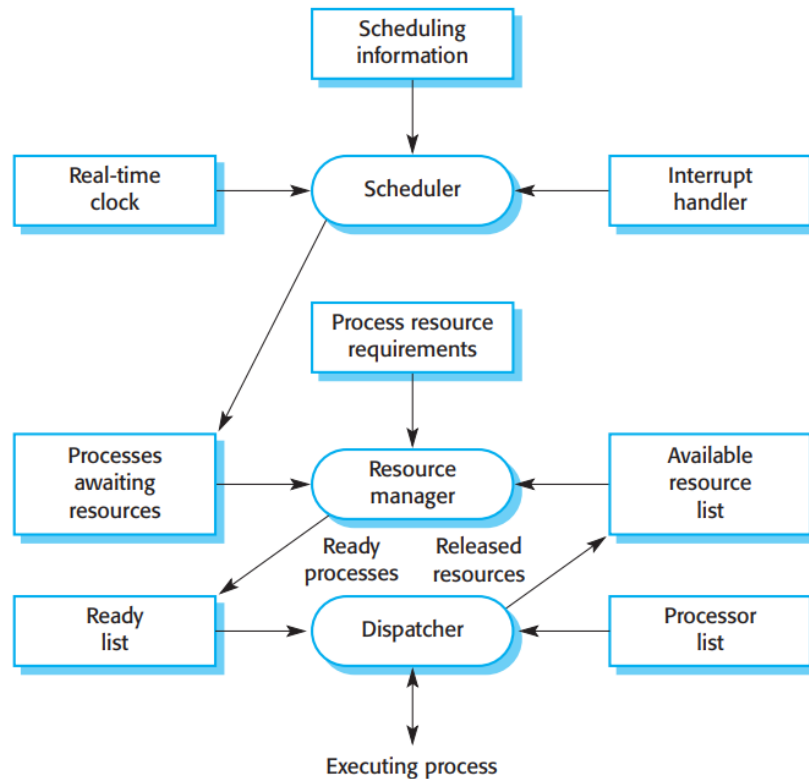


Figure 7.10 Components of a Real-time Operating System

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

Real-time operating systems for large systems, such as process control or telecommunication systems, may have additional facilities, namely, disk storage management, fault management facilities that detect and report system faults, and a configuration manager that supports the dynamic reconfiguration of real-time applications (Sommerville, n.d.).

Process Management

Real-time systems have to handle external events quickly and, in some cases, meet deadlines for processing these events. The event-handling processes must therefore be scheduled for execution in time to detect the event. They must also be allocated sufficient processor resources to meet their deadline. The process manager in an RTOS is responsible for choosing processes for execution, allocating processor and memory resources, and starting and stopping process execution on a processor (Sommerville, n.d.).

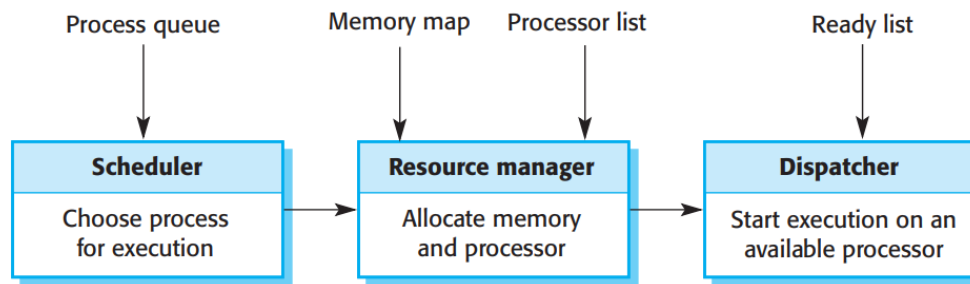


Figure 7.11 RTOS Actions Required to Start a Process

Source: Software Engineering, Tenth Edition, Ian Sommerville, 2016

The process manager has to manage processes with different priorities. For some stimuli, such as those associated with certain exceptional events, it is essential that their processing should be completed within the specified time limits. Other processes may be safely delayed if a more critical process requires service. Consequently, the RTOS has to be able to manage at least two priority levels for system processes (Sommerville, n.d.):

1. Clock level This level of priority is allocated to periodic processes.
2. Interrupt level This is the highest priority level. It is allocated to processes that need a very fast response. One of these processes will be the real-time clock process. This process is not required if interrupts are not supported in the system.

A further priority level may be allocated to background processes (such as a self-checking process) that do not need to meet real-time deadlines. These processes are scheduled for execution when processor capacity is available. Periodic processes must be executed at specified time intervals for data acquisition and actuator control. In most real-time systems, there will be several types of periodic process.

Using the timing requirements specified in the application program, the RTOS arranges the execution of periodic processes so that they can all meet their deadlines. The actions taken by the operating system for periodic process management are shown in Figure 7.11 (Sommerville, n.d.).

The scheduler examines the list of periodic processes and selects a process to be executed. The choice depends on the process priority, the process periods, the expected execution times, and the deadlines of the ready processes. Sometimes two processes with different deadlines should be executed at the same clock tick. In such a situation, one process

must be delayed. Normally, the system will choose to delay the process with the longest deadline. Processes that have to respond quickly to asynchronous events may be interruptdriven.

The computer's interrupt mechanism causes control to transfer to a predetermined memory location. This location contains an instruction to jump to a simple and fast interrupt service routine. The service routine disables further interrupts to avoid being interrupted itself. It then discovers the cause of the interrupt and initiates, with a high priority, a process to handle the stimulus causing the interrupt. In some high-speed data acquisition systems, the interrupt handler saves the data that the interrupt signaled was available in a buffer for later processing. Interrupts are then enabled again, and control is returned to the operating system (Sommerville, n.d.).

At any one time several processes, all with different priorities, could be executed. The process scheduler implements system-scheduling policies that determine the order of process execution. There are two commonly used scheduling strategies (Sommerville, n.d.):

1. **Non-preemptive Scheduling.** After a process has been scheduled for execution, it runs to completion or until it is blocked for some reason, such as waiting for input. This can cause problems if there are processes with different priorities and a high-priority process has to wait for a low-priority process to finish.
2. **Preemptive Scheduling.** The execution of an executing process may be stopped if a higher-priority process requires service. The higher-priority process preempts the execution of the lower-priority process and is allocated to a processor. Within these strategies, different scheduling algorithms have been developed.

These include round-robin scheduling, where each process is executed in turn; rate monotonic scheduling, where the process with the shortest period (highest frequency) is given priority; and shortest deadline first scheduling, where the process in the queue with the shortest deadline is scheduled (Burns and Wellings 2009).

Information about the process to be executed is passed to the resource manager. The resource manager allocates memory and, in a multiprocessor system, also adds a processor to this process. The process is then placed on the "ready list," a list of processes that are ready for execution. When a processor finish executing a process and becomes available, the dispatcher is invoked. It scans the ready list to find a process that can be executed on the available processor and starts its execution.



Assessment Task

Part I. Read the directions carefully. Answer all the questions based on your understanding from the lessons in this module.

A. Multiple Choice. Underline correct answer

1. This initial system engineering activity develops the concept of the type of system that is required.
 - a. Conceptual Design
 - b. Procurement of Acquisition
 - c. Development
 - d. Operation

2. During this stage, the conceptual design is further developed so that information is available to make decisions about the contract for the system development.
 - a. Conceptual Design
 - b. Procurement of Acquisition
 - c. Development
 - d. Operation

3. During this stage operational processes are defined, and the training courses for system users are designed.
 - a. Conceptual Design
 - b. Procurement of Acquisition
 - c. Development
 - d. Operation

4. During this stage the planned operational processes usually then have to change to reflect the real working environment where the system is used.
 - a. Conceptual Design

- b. Procurement of Acquisition
 - c. Development
 - d. Operation
5. They are involved because new air traffic management systems usually have to be installed in a new building.
- a. Architects and Civil Engineers
 - b. Electrical and Mechanical Engineers
 - c. Electronic Engineers
 - d. Ergonomists
6. They are involved to specify and maintain the power and air conditioning.
- a. Architects and Civil Engineers
 - b. Electrical and Mechanical Engineers
 - c. Electronic Engineers
 - d. Ergonomists
7. They are concerned with computers, radars, and other equipment.
- a. Architects and Civil Engineers
 - b. Electrical and Mechanical Engineers
 - c. Electronic Engineers
 - d. Ergonomists
8. They design the controller workstations and software engineers, and user interface designers are responsible for the software in the system.
- a. Architects and Civil Engineers
 - b. Electrical and Mechanical Engineers
 - c. Electronic Engineers
 - d. Ergonomists
9. A new system may mean that people have to change the way that they work.
- a. Process Changes
 - b. Job Changes
 - c. Organizational Policies
 - d. Organizational Politics

10. New systems may deskill the users in an environment or cause them to change the way they work.

Process Changes

Job Changes

Organizational Policies

Organizational Politics

11. The proposed system may not be completely consistent with organizational policies

Process Changes

Job Changes

Organizational Policies

Organizational Politics

12. If an organization reorganizes itself into a different structure, this may affect the requirements and use of the system.

Process Changes

Job Changes

Organizational Policies

Organizational Politics

13. Its examples are reliability, performance, safety, and security.

Emergent Properties

Functional Emergent Properties

Non-functional Emergent Properties

None of the Choices

14. When the purpose of a system only emerges after its components are integrated.

Emergent Properties

Functional Emergent Properties

Non-functional Emergent Properties

None of the Choices

15. They only emerge once the system components have been integrated.

Emergent Properties

Functional Emergent Properties

Non-functional Emergent Properties

None of the Choices

B. Fill in the Blanks. Give the exact and correct answer. Read the statement carefully, giving an incomplete and more than the desired answer will mean incorrect.

1. Its _____, depend on both the properties of individual components and their interactions or relationships.
2. _____ failure can sometimes generate spurious signals that are outside the range of inputs expected by the software.
3. _____ reflects how easy it is to fix a problem with the system once it has been discovered
4. The _____ of the system is a complex property that cannot be easily measured.
5. _____ reflects how easy it is to use the system.
6. The _____ of a system depends on how the component assemblies are arranged.
7. _____ systems are nondeterministic partly because they include people and partly because changes to the hardware, software, and data in these systems are so frequent.
8. _____ is often seen as a bad thing, and it is felt that designers should try to avoid nondeterministic behavior wherever possible.
9. The judgment of _____ is not usually made against the original reasons for procuring and developing the system.
10. In the _____ phase, you take that initial idea, investigate its feasibility, and develop it to create an overall vision of a system that could be developed.
11. There is an obvious _____ between conceptual design and requirements engineering
12. Concept _____ is the first stage of the process where trying to refine an initial statement of needs and work out what type of system would be best to meet the needs of system stakeholders.
13. In the system proposal development stage, the conceptual _____ team set out their ideas.
14. System _____ or system acquisition is a process whose outcome is a decision to buy one or more systems from system suppliers.

15. _____ complex systems are usually engineered using a mixture of off-the-shelf and specially built components.

C. True or False. Write T if you agreed that the statement given is true and F if you think the statement is false.

1. Organizations often have an approved and recommended set of application software that has been checked by the IT department.
2. Off-the-shelf components do not usually match requirements exactly, unless the requirements have been written with these components in mind.
3. When a system is to be built specially, the specification of requirements is part of the contract for the system being acquired.
4. For complex hardware/software systems, it may be necessary to use a group of suppliers, each with a different type of expertise.
5. Decisions made at the procurement stage of the systems engineering process are critical for later stages in that process.
6. For public sector systems in particular, there are no detailed rules and regulations that affect the procurement of systems.
7. For application systems that require change or for custom systems, there is no usual a contract negotiation period when the customer and supplier negotiate the terms and conditions for development of the system.
8. Complex sociotechnical systems are usually developed “in house” by the buyer of the system.
9. Plan-driven processes are used in systems engineering because same elements of the system are independently developed
10. Requirements gathering overlaps significantly with the requirements engineering process.



Summary

An embedded software system is part of a hardware/software system that reacts to events in its environment. The software is “embedded” in the hardware. Embedded systems are normally real-time systems. A real-time system is a software system that must respond

to events in real time. System correctness does not just depend on the results it produces, but also on the time when these results are produced.

Real-time systems are usually implemented as a set of communicating processes that react to stimuli to produce responses. State models are an important design representation for embedded real-time systems. They are used to show how the system reacts to its environment as events trigger changes of state in the system.

Several standard patterns can be observed in different types of embedded system. These include a pattern for monitoring the system's environment for adverse events, a pattern for actuator control, and a data-processing pattern.

Designers of real-time systems have to do a timing analysis, which is driven by the deadlines for processing and responding to stimuli. They have to decide how often each process in the system should run and the expected and worst-case execution time for processes. A real-time operating system is responsible for process and resource management. It always includes a scheduler, which is the component responsible for deciding which process should be scheduled for execution (Sommerville, n.d.).



References

A. eBook

Advanced Software Engineering, Ian Sommerville, c2016, Retrieved from:
<https://www.advsofteng.com/about.html>

Architecting Principles for Systems-of-Systems, Systems Engineering, Maier, M. W. 1998. (4):
267–284. doi:10.1002/(SICI)1520-6858(1998)1:43.0.CO;2-D.

Real-time Systems: Design Principles for Distributed Embedded Applications, 2nd Ed. (H.
Kopetz, Springer, 2013).

A. Online Resources

Component Software: Beyond Object-Oriented Programming, 2nd Ed. Harlow, Szyperski, C. 2002, UK: Addison Wesley. Retrieved from:

<https://dl.acm.org/doi/book/10.5555/515228>

Definition of a Software Component and Its Elements. In Component-Based Software Engineering, G. T. Heineman and W. T. Council. Retrieved from:

<https://dl.acm.org/doi/10.5555/379381.379438>

Object-Oriented Application Frameworks, Fayad, M. E., and D. C. Schmidt. 1997, Comm. ACM 40 (10): 32–38. Retrieved from:

<https://dl.acm.org/doi/10.1145/262793.262798>

Representational State Transfer. Architectural Styles and the Design of Network Based Software Architecture. Fielding, R. 2000. Retrieved from:

https://www.ics.uci.edu/~fielding/pubs/. . ./fielding_dissertation.pdf

Software Component Models, IEEE Trans. on Software Eng. 33 (10): 709–724. Lau, K-K., and Z. Wang. 2007. Retrieved from: doi:10.1109/TSE.2007.70726.

Trends in Embedded Software Engineering. (IEEE Software, 26 (3), May–June 2009)

Retrieved from: <http://dx.doi.org/10.1109/MS.2009.80>

- END OF THE MIDTERM MODULE -

CHECK YOUR EXAM SCHEDULE FOR THIS COURSE.

DO NOT FORGET TO TAKE THE EXAM AS SCHEDULED.

THANK YOU AND GOD BLESS